



Universidad Autónoma de Baja California

Programación Orientada a Objetos

Programación Orientada a Objetos

Unidad II: Modularidad y jerarquía

Profesor: MC. Itzel Barriba Cázares

Relaciones entre clases

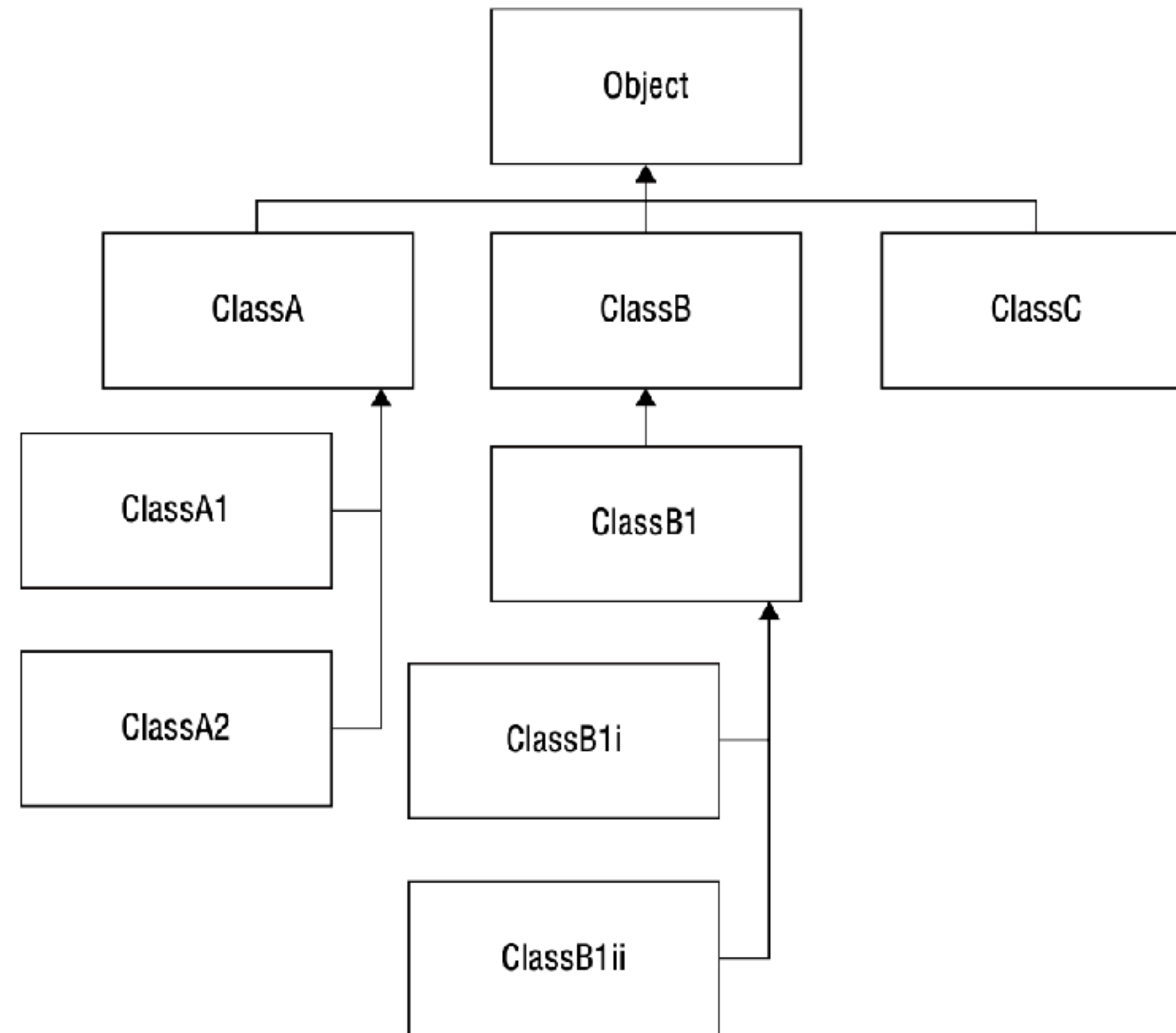
- En POO las relaciones entre clases son fundamentales para modelar el comportamiento y la estructura de un programa permitiendo una mayor reutilización y mantenibilidad del código.

Herencia

- ▶ La **herencia** es uno de los tres principios fundamentales de la POO, permite la creación de clasificaciones jerárquicas.
- ▶ La herencia permite que una nueva clase **herede** una clase existente.
- ▶ En el mundo real, se pueden encontrar muchos objetos que son versiones especializadas de otros objetos más generales.

La clase object

- ▶ Las clases de java están organizadas en una **estructura de herencia jerárquica**.
- ▶ La **clase Object** es la superclase de más alto nivel que **no tiene una clase padre** encima.
- ▶ Todas las demás clases son subclases de Object o subclases de subclases de Object.



Herencia y la relación “es un”

- ▶ Cuando un objeto es una versión especializada de otro objeto, existe una relación “es un” entre ellos.
- ▶ En una relación “es un” un objeto de una subclase puede ser tratado como un objeto de su superclase.

Herencia y la relación “es un”

- ▶ Ejemplos:

- ▶ El carro es un Vehículo



- ▶ La naranja es una fruta



- ▶ El cirujano es un doctor

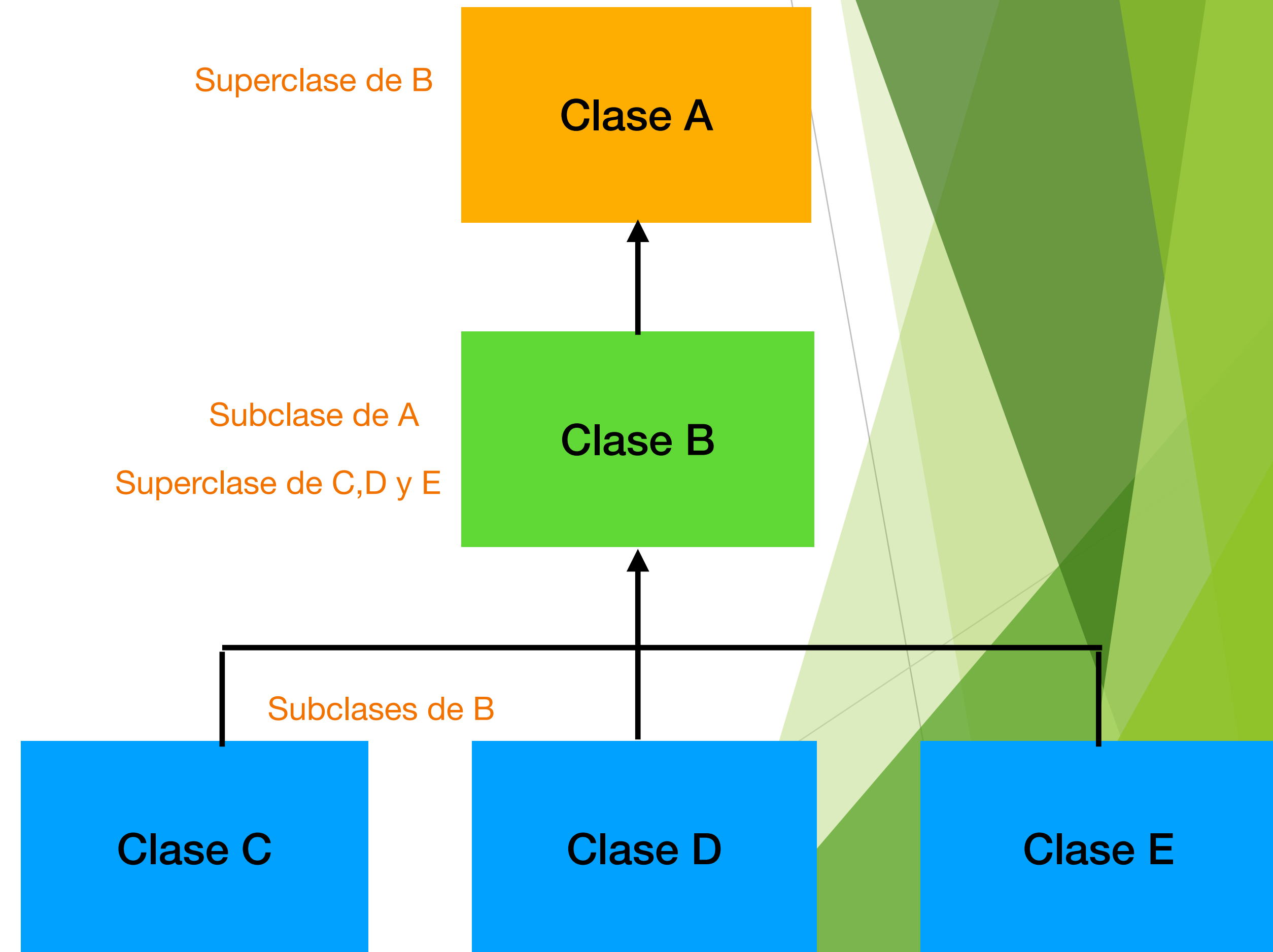


- ▶ El perro es un Animal



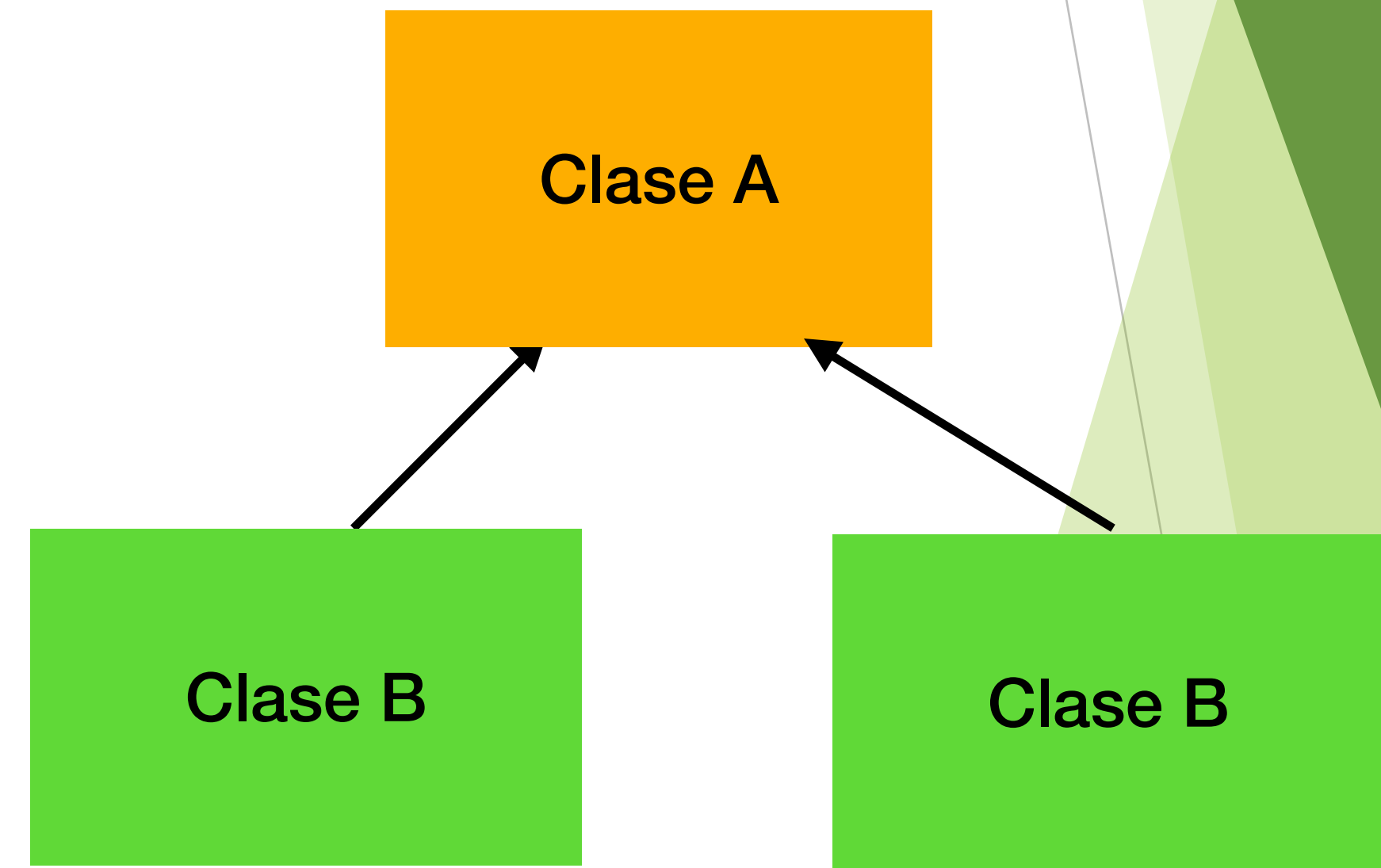
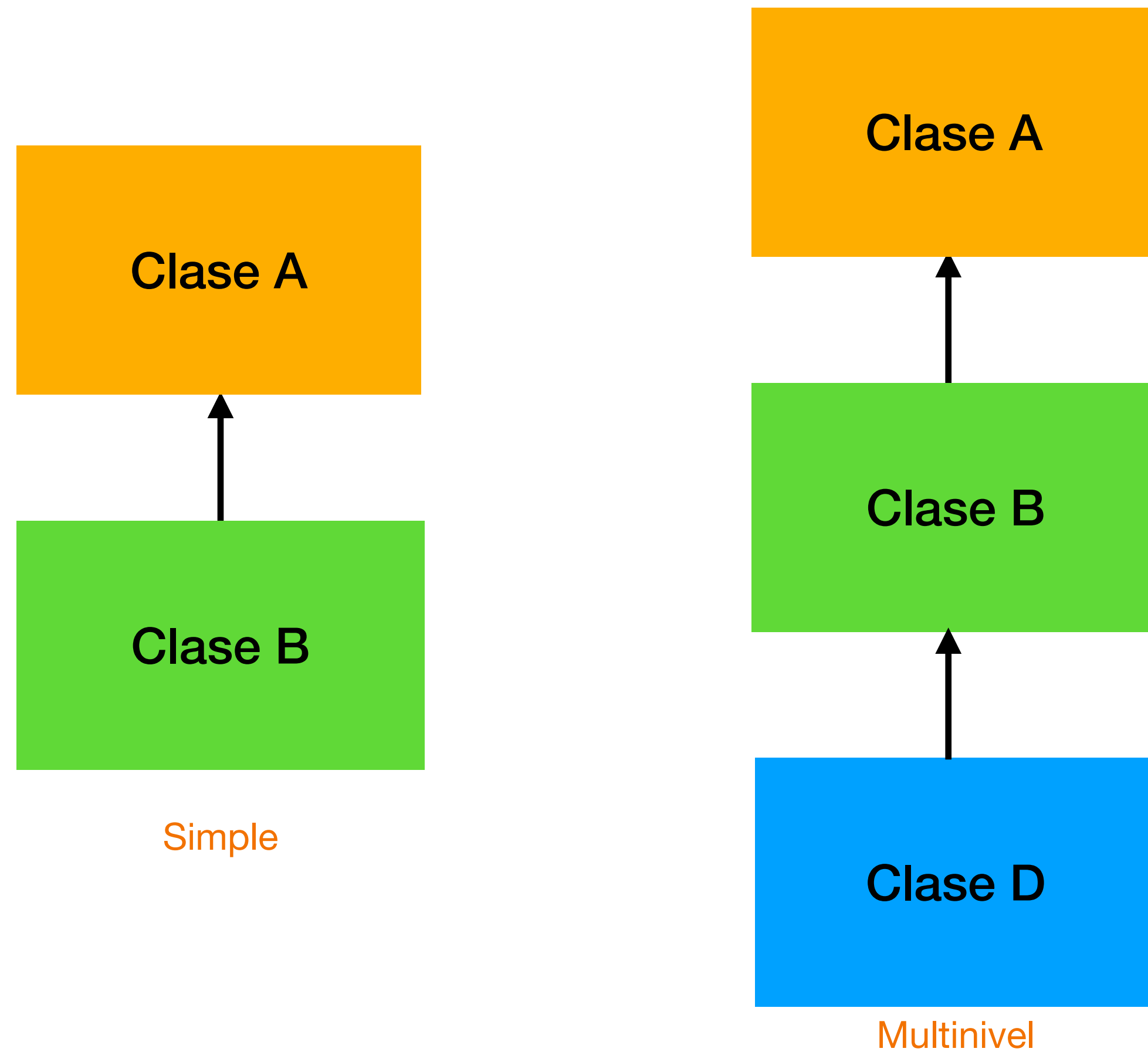
Herencia

- La herencia implica una superclase y una subclase.



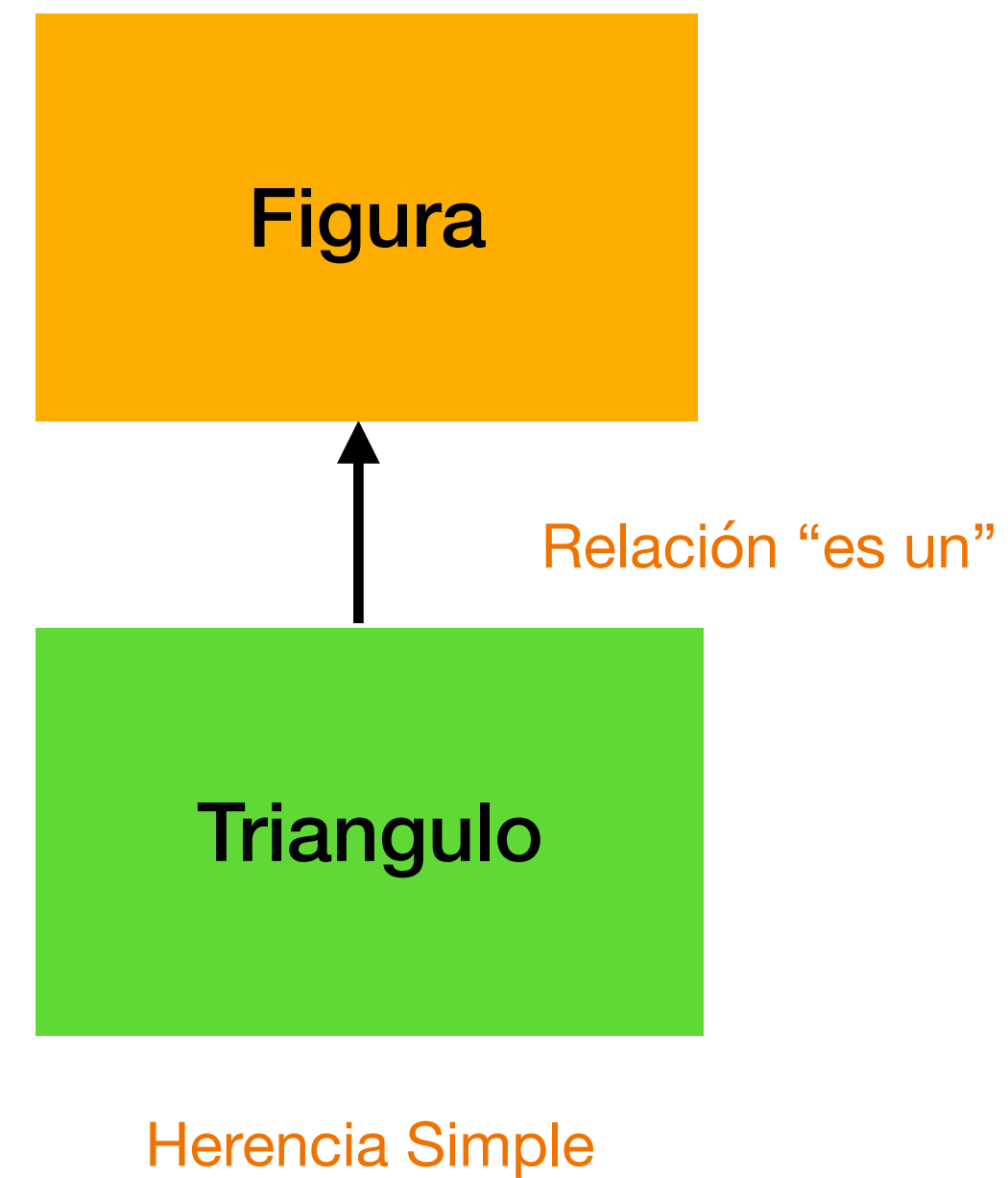
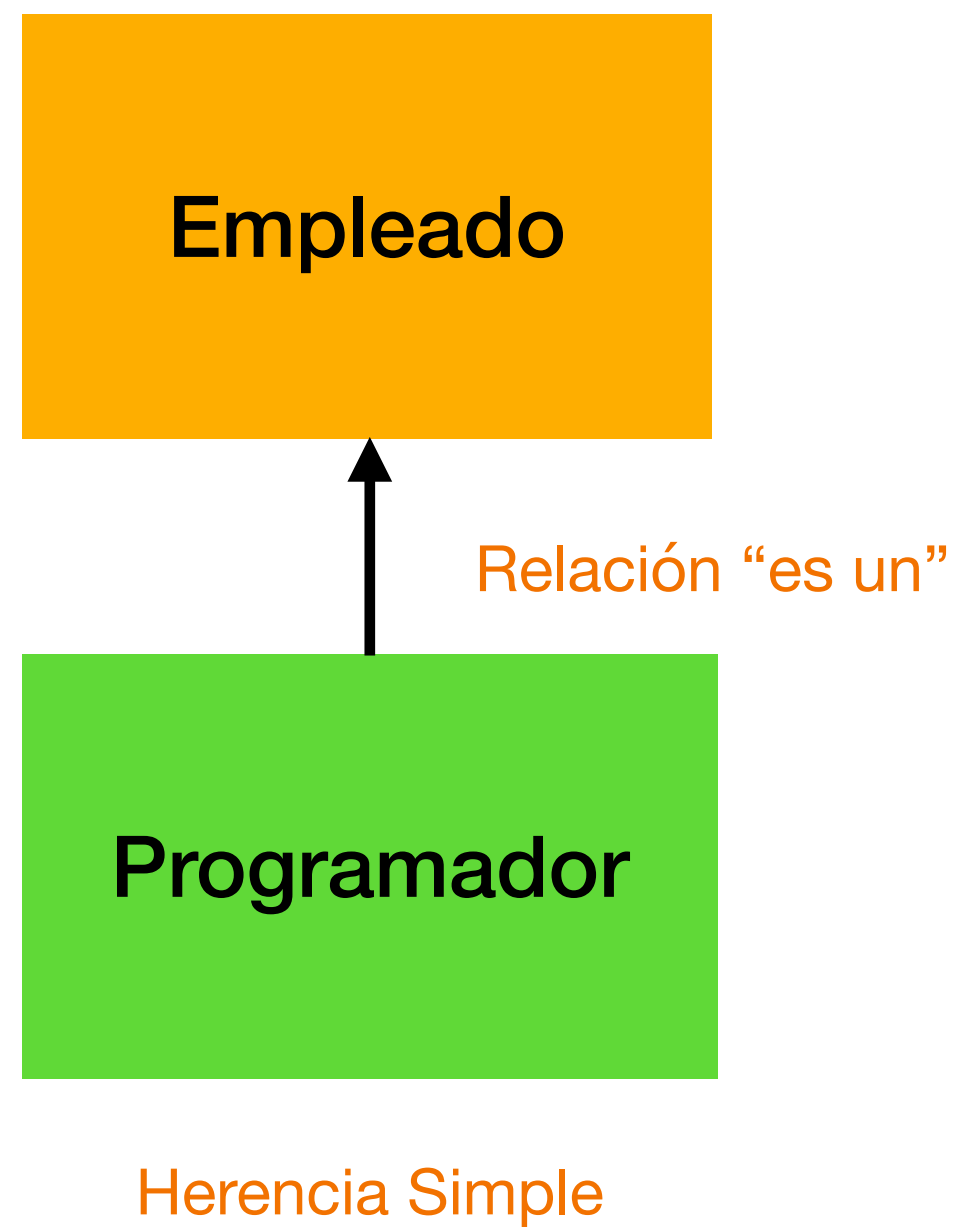
Herencia

- Tipos de herencia en Java: puede haber tres tipos de herencia en Java

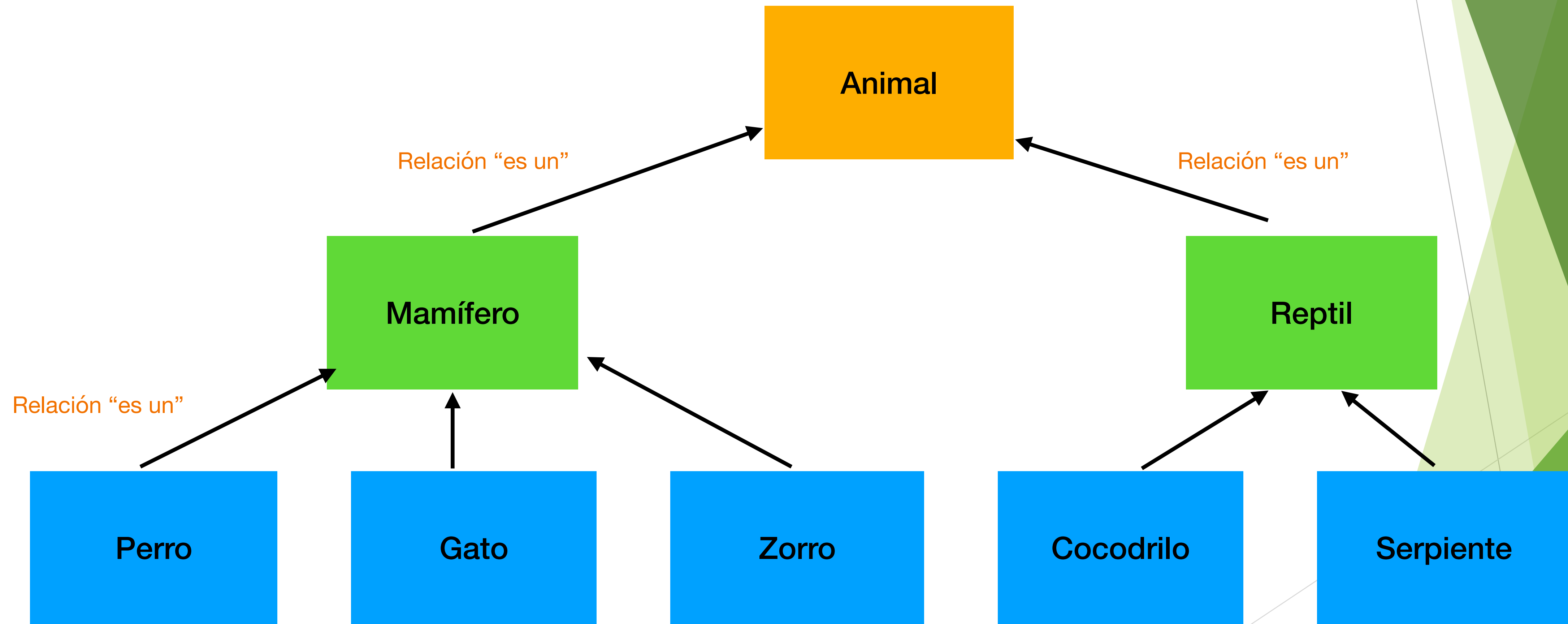


Jerárquica

Herencia simple



Herencia de la clase Animal

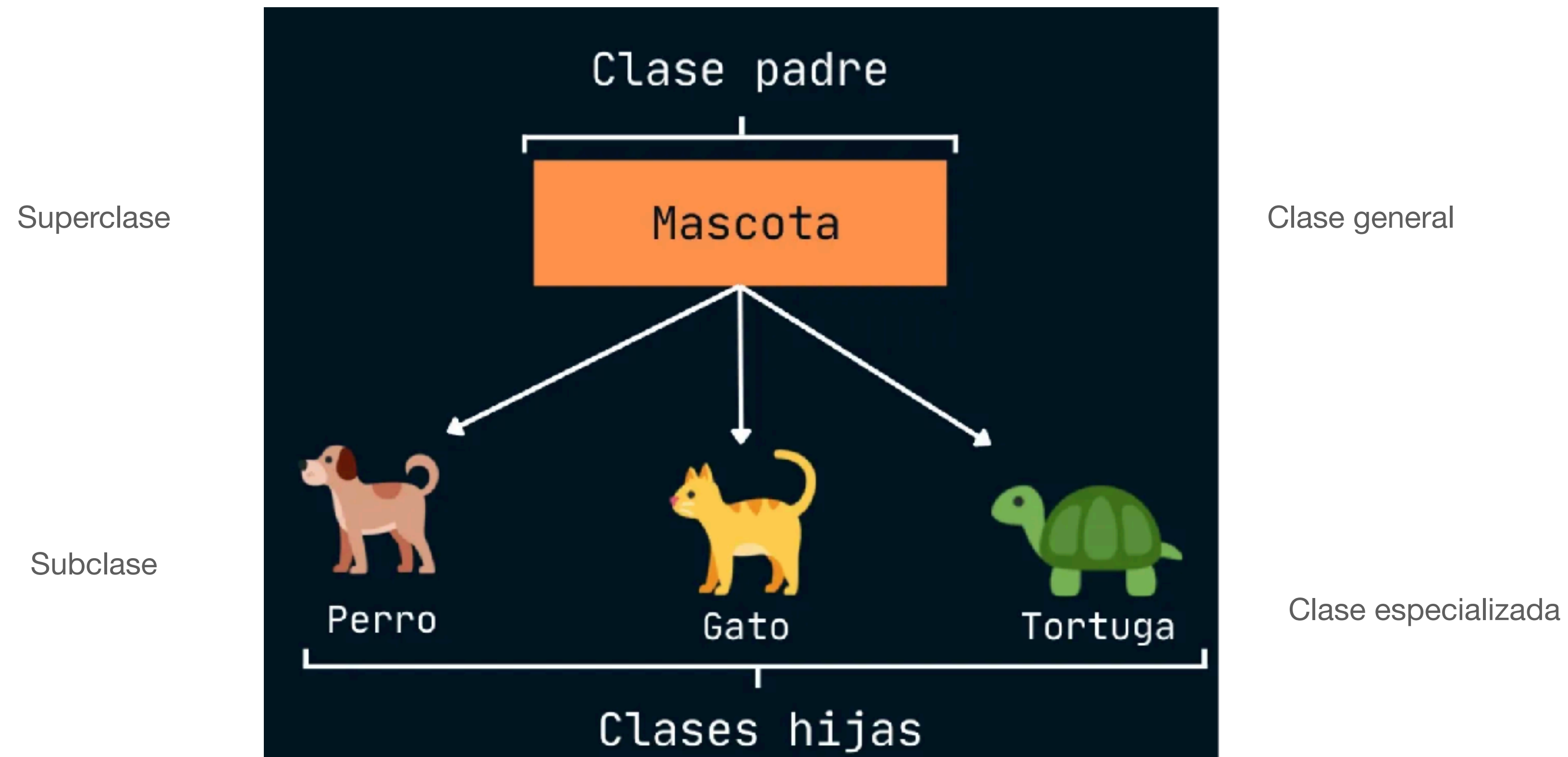


Herencia y la relación “es un”

- Cuando existe una relación “es un” entre objetos, significa que el objeto especializado tiene todas las características del objeto general, además de características que lo hacen especial



Herencia



Ejemplos de herencia

Superclase	Subclases
Estudiante	
Figura	
Prestamo	
Empleado	
CuentaBanco	

Ejemplos de herencia

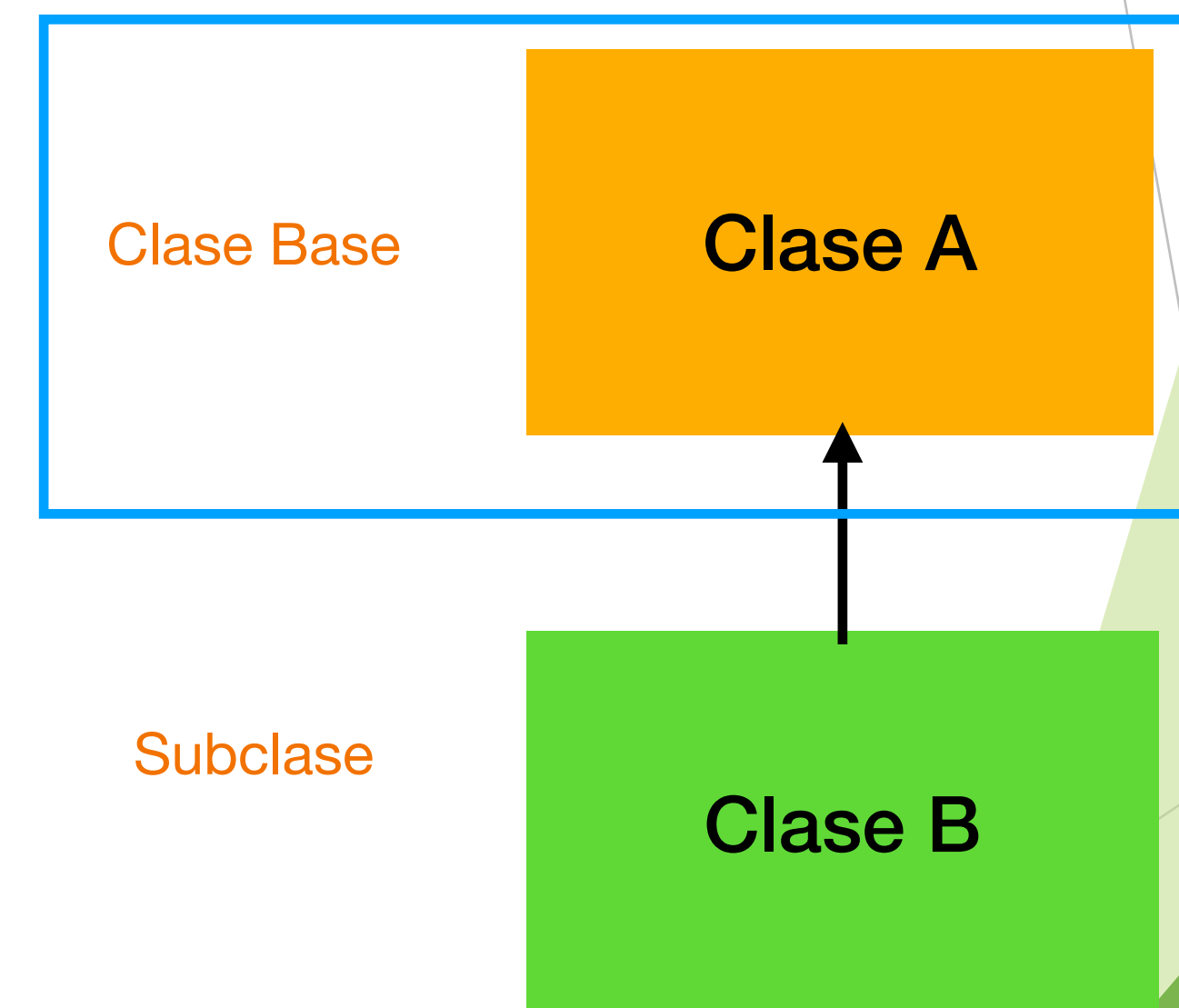
Superclase	Subclases
Estudiante	Graduado, Universitario
Figura	Circulo, Triangulo, Rectángulo, Esfera, Cubo
Prestamo	PrestamoCarro, PrestamoHipotecario, PrestamoRemodelacion
Empleado	Academico, Administrativo
CuentaBanco	CuentaAhorro, CuentaCheques

Herencia simple: Clase Base

- La **clase Base** define los atributos y métodos. El siguiente ejemplo crea una superclase llamada A y una subclase B.

//Creación de una superclase

```
public class A {  
    int i, j;  
  
    void mostrarij() {  
        System.out.println("i: " + i + " j: " + j);  
    }  
}
```



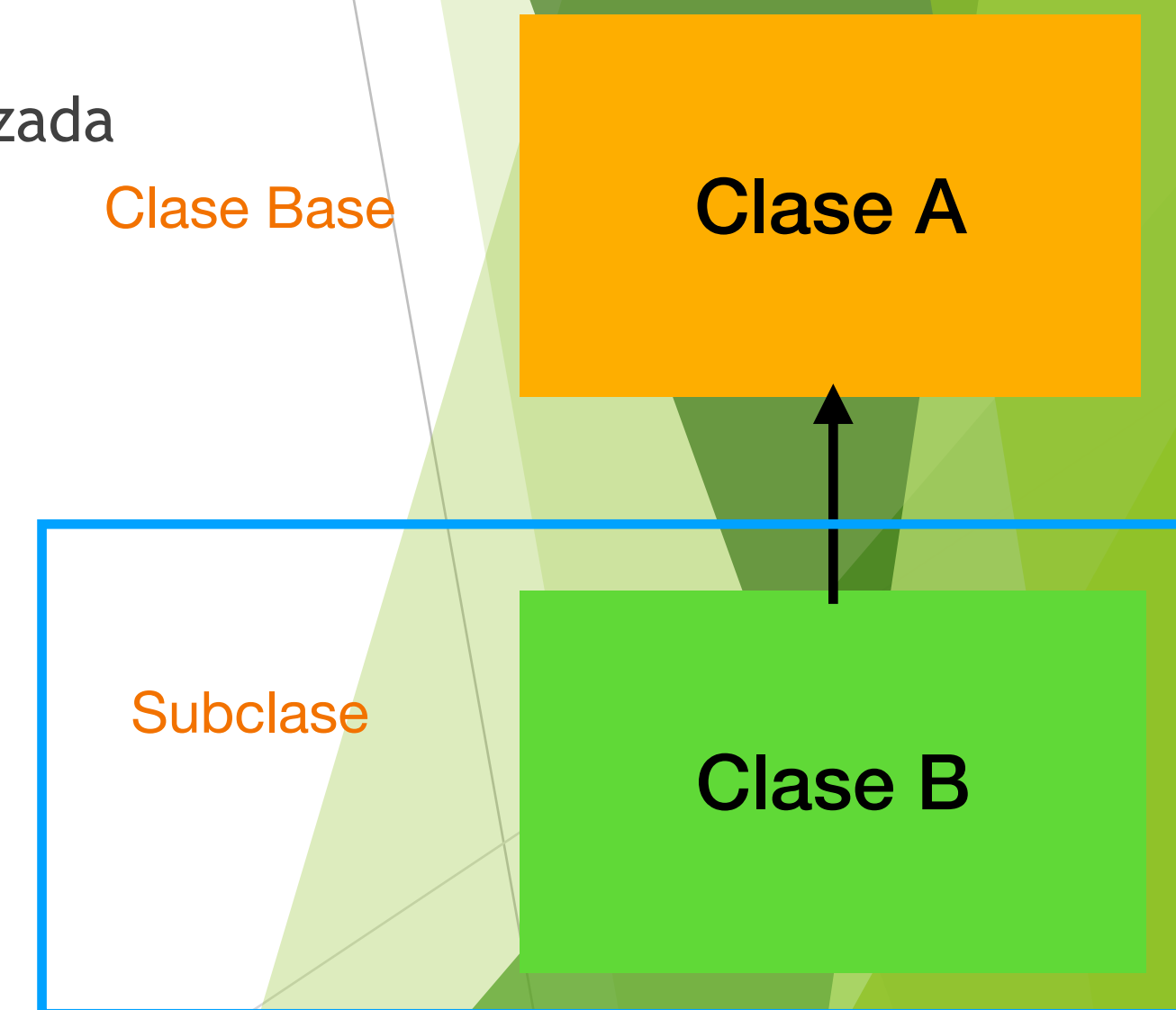
Herencia simple

- ▶ La clase derivada o subclase **hereda atributos y métodos** de la superclase, es decir puede **acceder** a los **miembros** de la clase base, sin que sea necesario reescribir ninguno de ellos.
- ▶ Para **heredar una clase**, se utiliza la palabra **extends**.
- ▶ Además puede agregar nuevos campos y métodos a la subclase, y eso la convierte en una versión especializada de la superclase.

```
//Subclase que hereda de la clase A
class B extends A {
    int k;

    void mostrark() {
        System.out.println("k: " + k);
    }
    void suma() {
        System.out.println("i+j+j: " + i+j+k);
    }
}
```

- ▶ Por ejemplo, un objeto de la clase A tiene relación “**es un**” de la clase B, por lo tanto tendrá los atributos i y j, y como parte de sus miembros de especialización, define el atributo k que tiene la clase B y como parte de sus acciones o comportamiento se define la suma.



Herencia simple: clase prueba

- La clase `prueba` donde se muestra el uso de herencia simple

```
public class HerenciaSimple {  
    public static void main(String[] args) {  
        A superOb = new A();  
        B subOb = new B();  
  
        //la superclase puede usarse sola  
        superOb.i = 10;  
        superOb.j = 20;  
        System.out.println("Contenido del objeto padre:");  
        superOb.mostrarij();  
        System.out.println();  
  
        //la subclase tiene acceso a todos los miembros de su superclase  
        subOb.i = 7;  
        subOb.j = 8;  
        subOb.k = 9;  
        System.out.println("Contenido del objeto hijo:");  
        subOb.mostrarij();  
        subOb.mostrark();  
        System.out.println();  
        System.out.println("Suma de i + j + k en subclaseOb: ");  
        subOb.suma();  
    }  
}
```

Salida del código

```
Contenido del objeto padre:  
i: 10 j: 20  
  
Contenido del objeto hijo:  
i: 7 j: 8  
k: 9  
  
Suma de i + j + k en subclaseOb:  
i+j+j: 789
```

Acceso a miembros

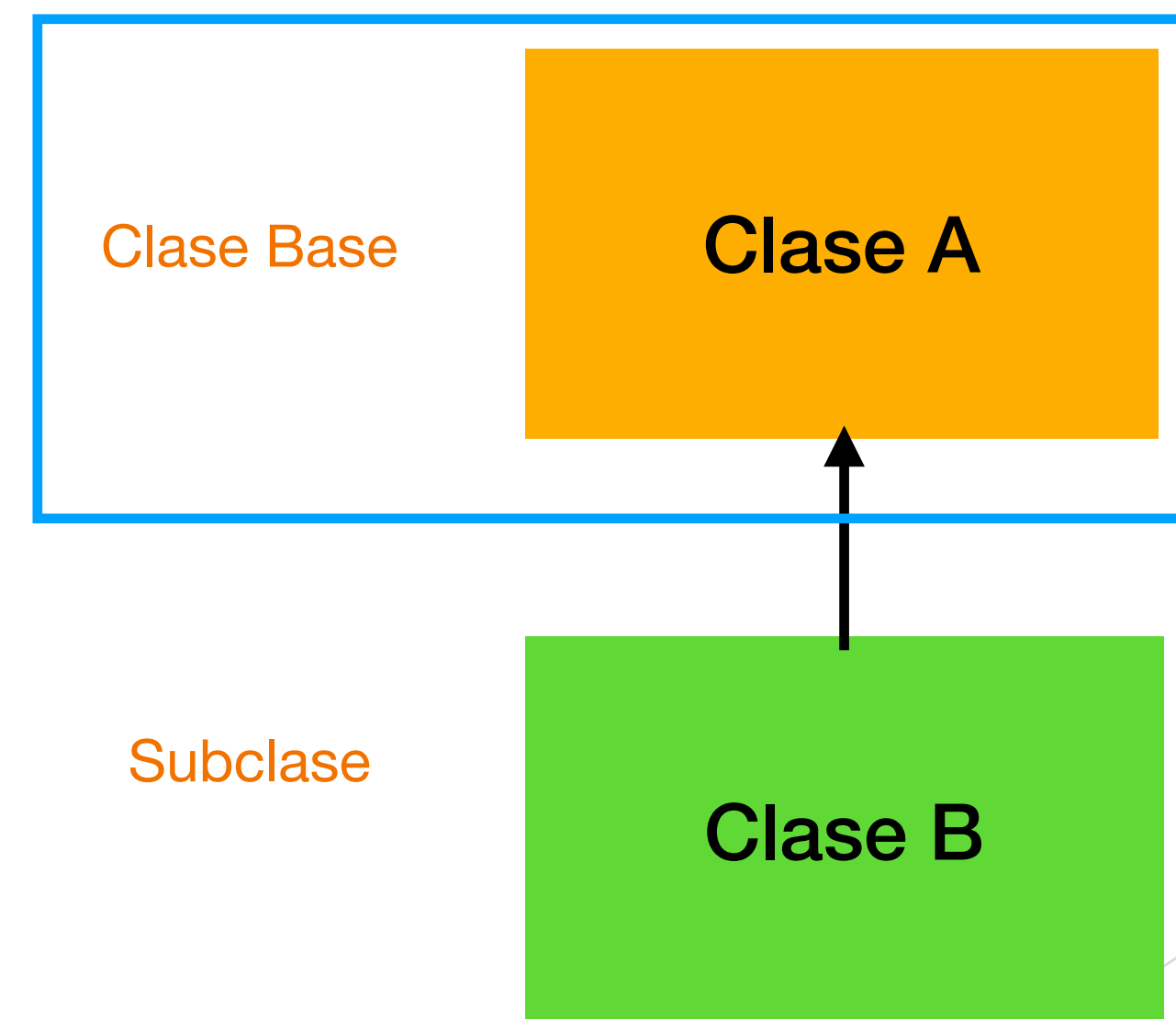
- ▶ Los miembros **públicos** pueden ser accedido desde cualquier parte del programa si se tiene una referencia al objeto.
- ▶ Los miembros **privados** solo pueden ser utilizados dentro de la misma clase.
- ▶ Las subclases **heredan únicamente los miembros no privados** de la superclase.
- ▶ Los miembros **protegidos** tienen un acceso intermedio: pueden ser usados por la clase, sus subclases y otras clases del mismo paquete.
- ▶ Las subclases pueden acceder directamente a los **miembros públicos y protegidos heredados** usando sus nombres.

Acceso a miembros y herencia

- ▶ Aunque una subclase incluye todos los miembros de una superclase, **no puede acceder a los miembros** de la superclase que se han declarado como **privados**.

//En una jerarquía de clases, los miembros privados permanecen privados a su clase

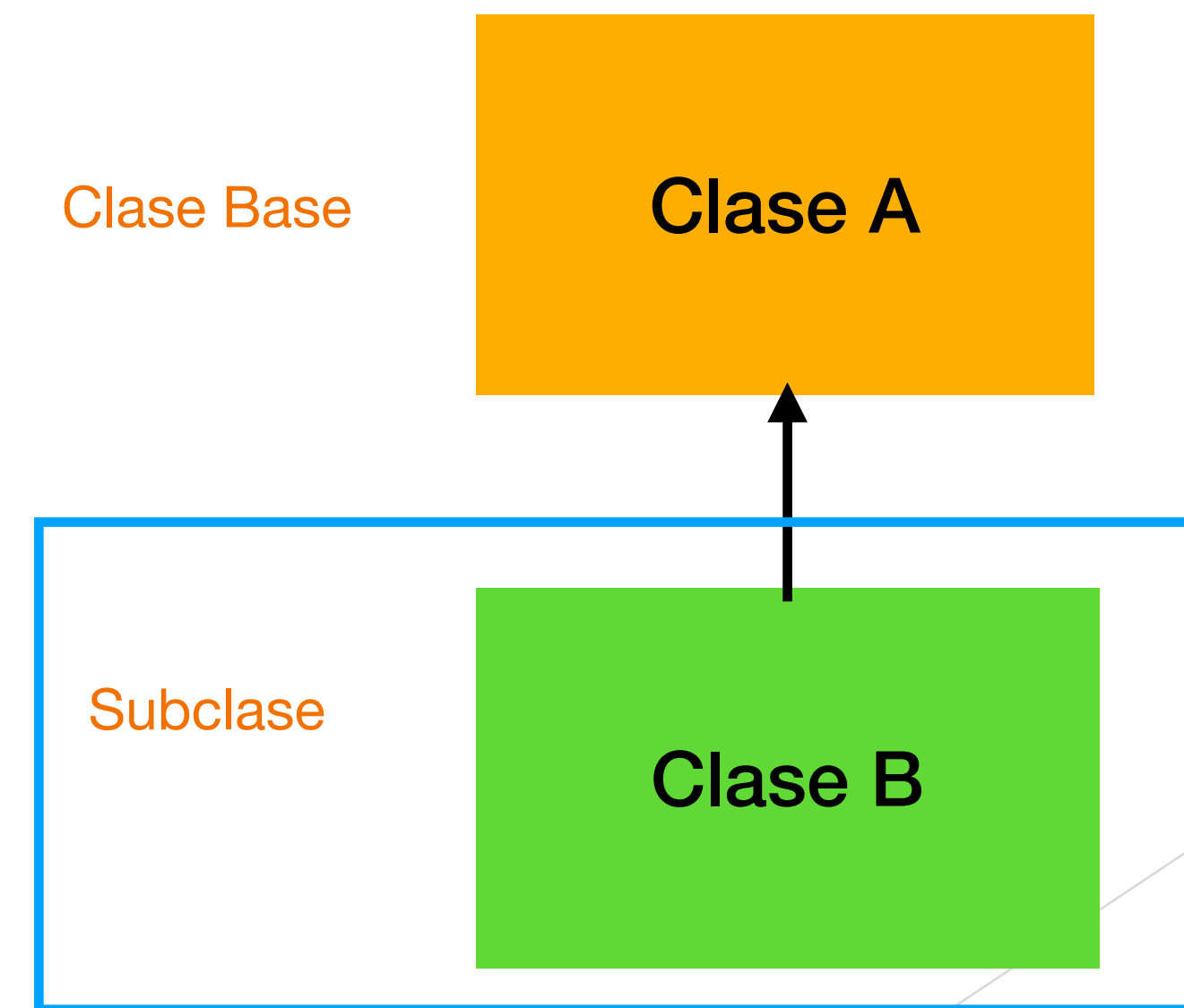
```
public class A {  
    int i;  
    private int j;  
  
    void setij(int x, int y) {  
        i = x;  
        j = y;  
    }  
}
```



Acceso a miembros y herencia

//El atributo j de la superclase no es accesible en la subclase

```
class BB extends A {  
    int total;  
  
    void suma() {  
        total = i + j;  
    }  
}
```



Acceso a miembros y herencia

//El atributo j de la superclase no es accesible en la subclase

```
class BB extends A {  
    int total;  
  
    void suma() {  
        total = i + j;  
    }  
}
```

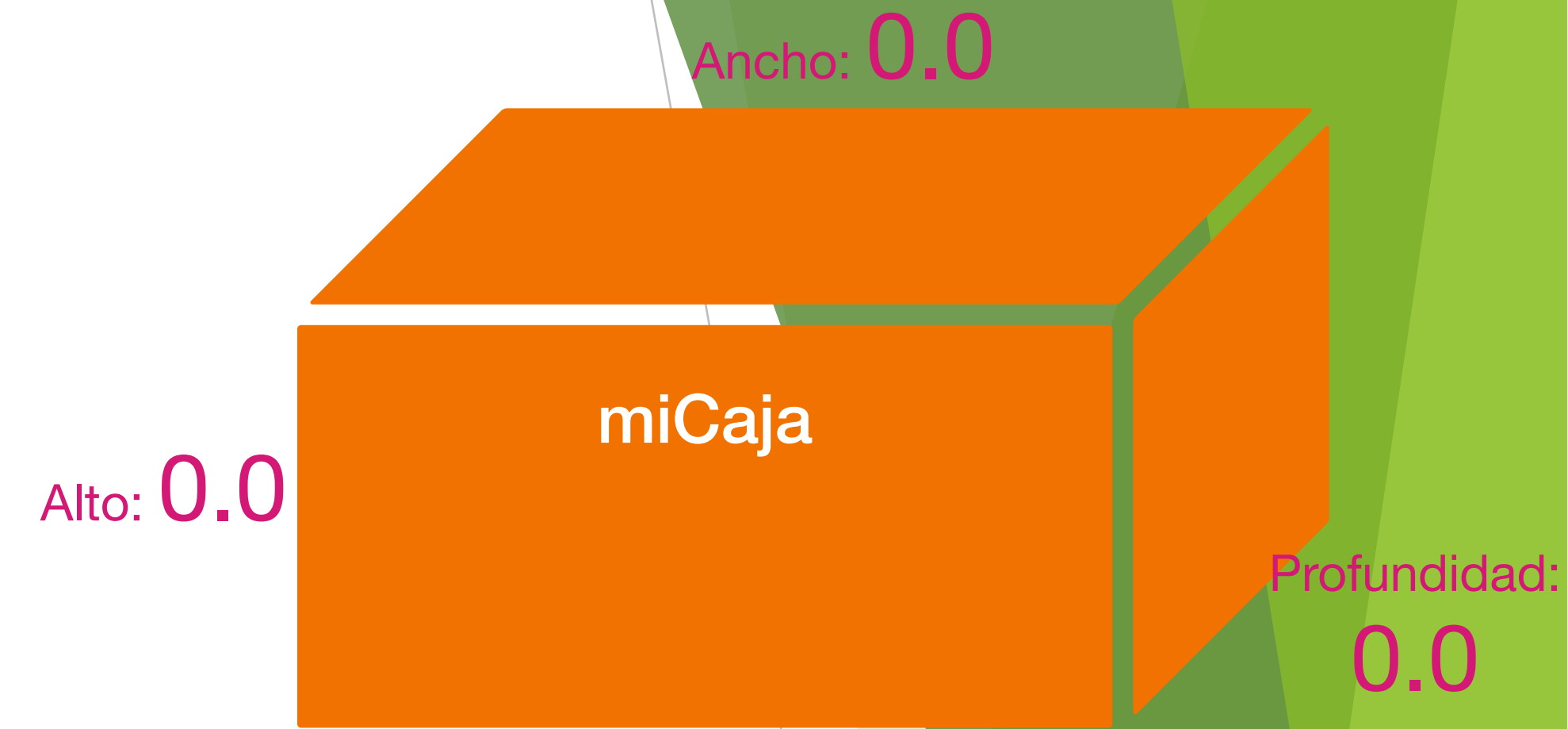
Salida del código

```
BB.java:7: error: j has private access in AA  
        total = i + j;  
                      ^  
1 error
```

Ejemplo 2: Herencia simple

```
// Este programa usa herencia, Clase base o superclase
public class Caja {
    double ancho;
    double altura;
    double profundidad;

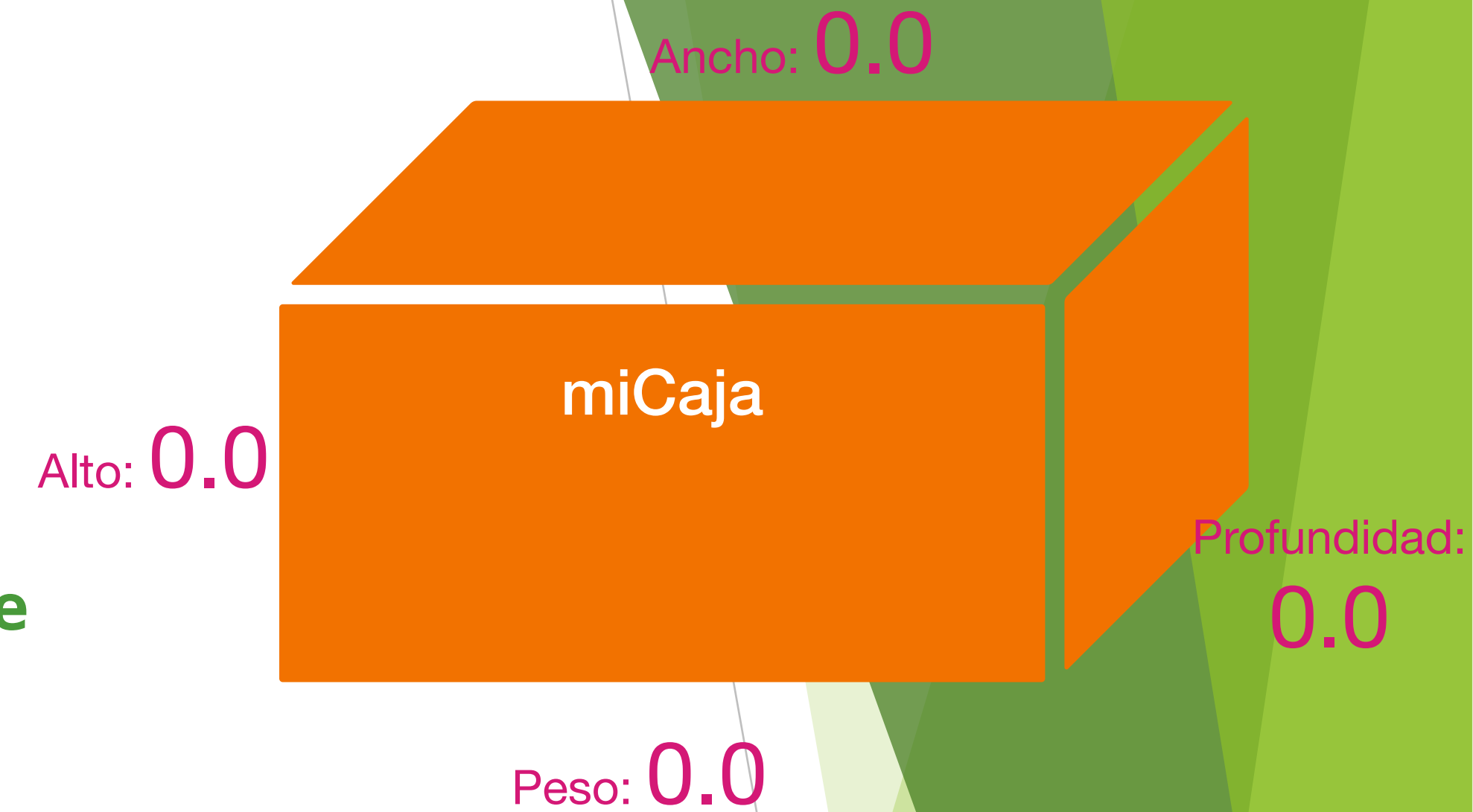
    // constructor usado cuando todas las dimensiones se especifican
    Caja(double w, double h, double d) {
        ancho = w;
        altura = h;
        profundidad = d;
    }
    // constructor usado cuando no se especifican las dimensiones
    Caja() {
        ancho = -1; // se inicializa en -1 todos los atributos
        altura = -1;
        profundidad = -1;
    }
    // constructor usado cuando la caja es un cubo
    Caja(double len) {
        ancho = altura = profundidad = len;
    }
    // calcula y regresa el volumen
    double volume() {
        return ancho * altura * profundidad;
    }
}
```



Ejemplo 2: Herencia simple

```
// Este programa usa herencia, Clase derivada o subclase
public class PesoCaja extends Caja {
    double peso; // peso de la caja

    // constructor para PesoCaja
    PesoCaja(double w, double h, double d, double m) {
        ancho = w;
        altura = h;
        profundidad = d;
        peso = m;
    }
}
```

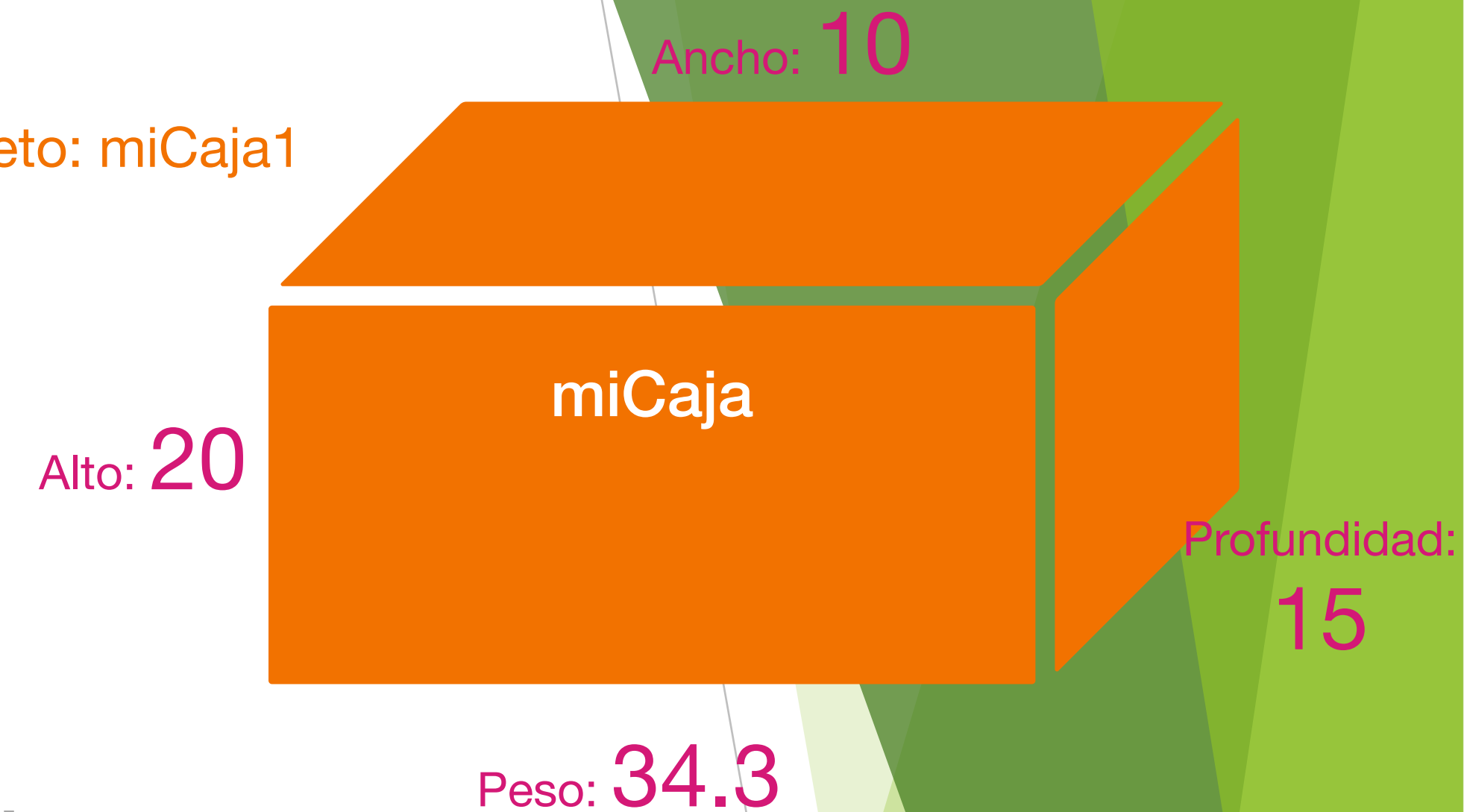


Ejemplo 2: Herencia simple

```
// Clase prueba para demostrar herencia simple
public class PruebaPesoCaja {
    public static void main(String[] args) {
        PesoCaja miCaja1 = new PesoCaja(10, 20, 15, 34.3);
        PesoCaja miCaja2 = new PesoCaja(2, 3, 4, 0.076);
        double vol;
        vol = miCaja1.volume();
        System.out.println("El volumen de miCaja1 es: " + vol);
        System.out.println("El peso de miCaja1 es: " + miCaja1.peso);
        System.out.println();

        vol = miCaja2.volume();
        System.out.println("El volumen de miCaja2 es " + vol);
        System.out.println("El volumen de miCaja2 es " + miCaja2.peso);
    }
}
```

Objeto: miCaja1



Salida del código

```
El volumen de miCaja1 es: 3000.0
El peso de miCaja1 es: 34.3

El volumen de miCaja2 es: 24.0
El peso de miCaja1 es: 0.076
```

Super()

- ▶ Se usa en constructores para invocar al constructor de la superclase.

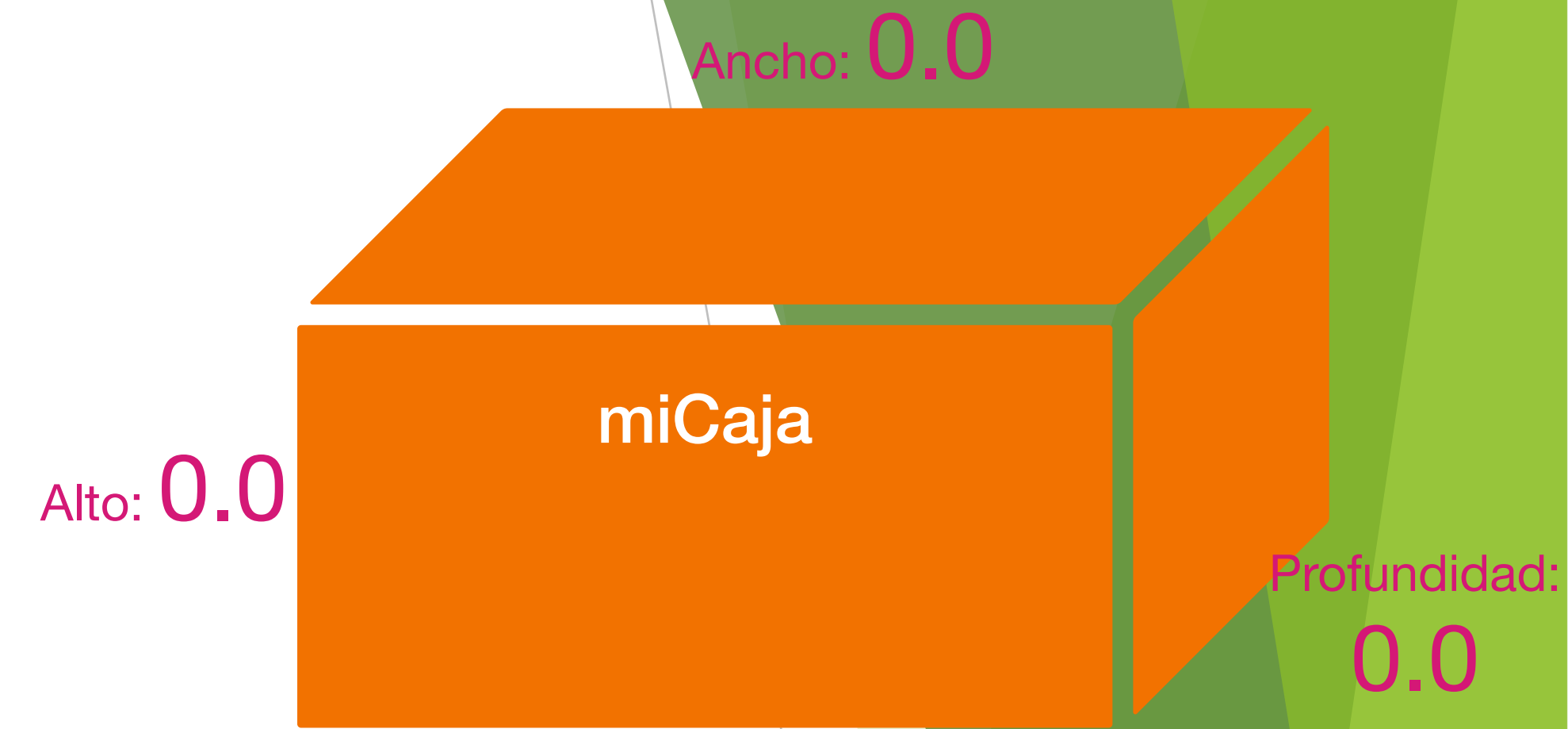
```
super(); //llamar al constructor base por defecto
```

- ▶ Solo puede utilizarse dentro del constructor de la superclase.
- ▶ Debe ser la **primera instrucción dentro del constructor de la subclase.**
- ▶ Si no se escribe, el compilador inserta automáticamente una llamada al constructor de clase base predeterminado sin argumentos.

Ejemplo 3: Herencia simple

```
// Este programa usa herencia, Clase base o superclase
public class Caja {
    double ancho;
    double altura;
    double profundidad;

    // constructor usado cuando todas las dimensiones se especifican
    Caja(double w, double h, double d) {
        ancho = w;
        altura = h;
        profundidad = d;
    }
    // constructor usado cuando no se especifican las dimensiones
    Caja() {
        ancho = -1; // se inicializa en -1 todos los atributos
        altura = -1;
        profundidad = -1;
    }
    // constructor usado cuando la caja es un cubo
    Caja(double len) {
        ancho = altura = profundidad = len;
    }
    // calcula y regresa el volumen
    double volume() {
        return ancho * altura * profundidad;
    }
}
```



Ejemplo 3: Herencia simple

// Este programa usa herencia, Clase derivada o subclase usando super

```
public class PesoCaja2 extends Caja {  
    double peso; // peso de la caja
```

```
    // constructor para PesoCaja
```

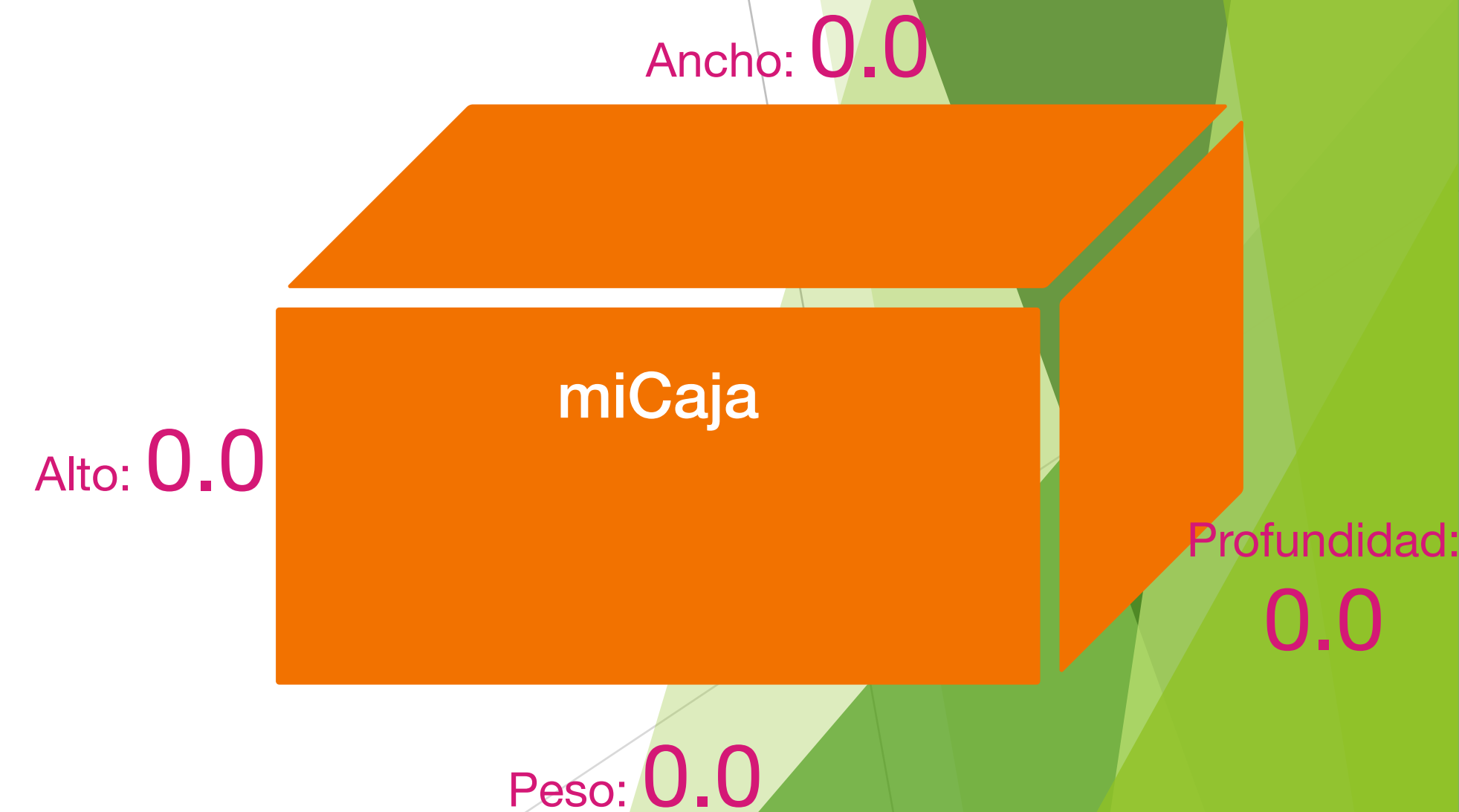
```
    PesoCaja2(double w, double h, double d, double m) {
```

```
        super(w,h,d);
```

```
        peso = m;
```

```
    }
```

```
}
```

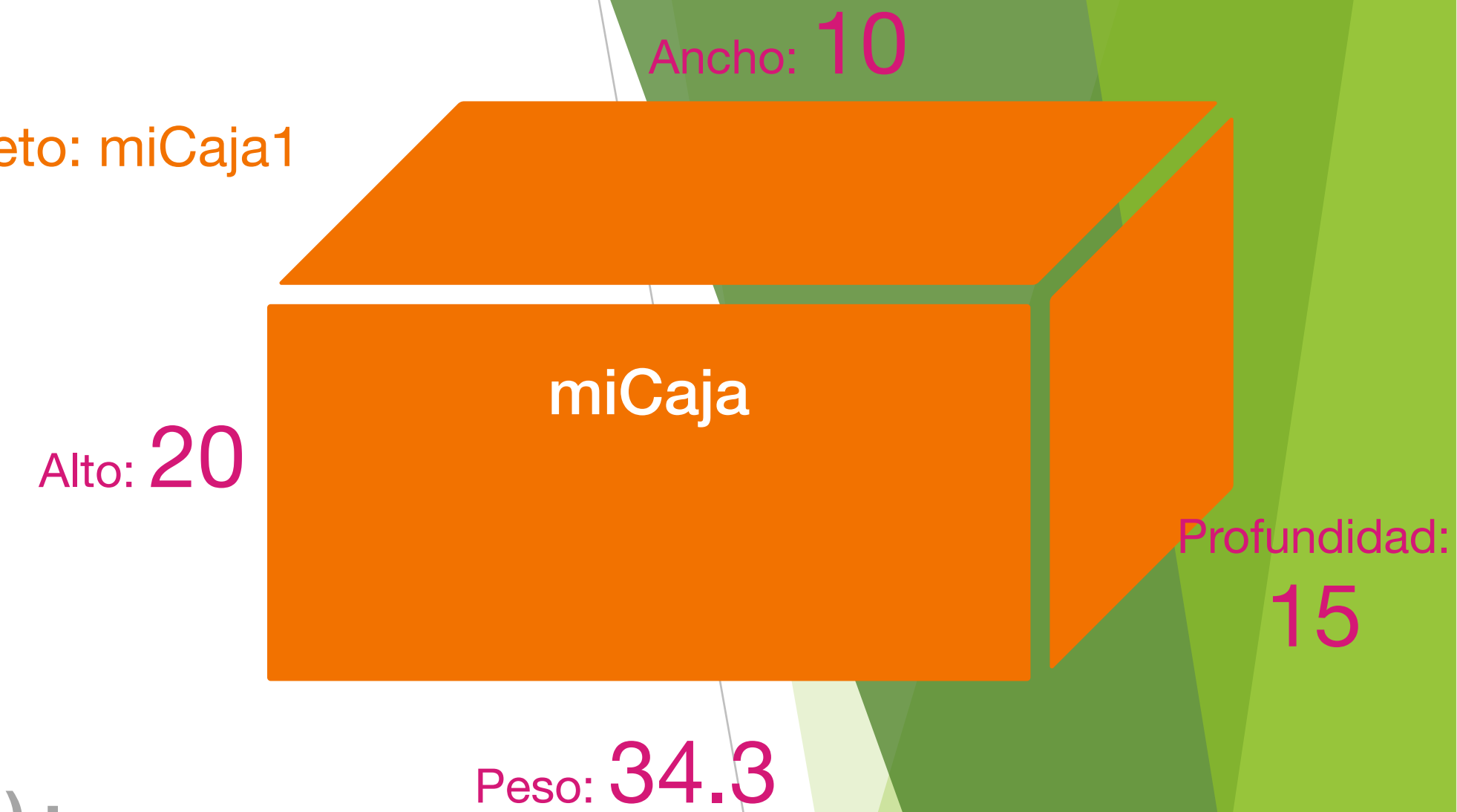


Ejemplo 3: Herencia simple

```
// Clase prueba para demostrar herencia simple
public class PruebaPesoCaja {
    public static void main(String[] args) {
        PesoCaja miCaja1 = new PesoCaja(10, 20, 15, 34.3);
        PesoCaja miCaja2 = new PesoCaja(2, 3, 4, 0.076);
        double vol;
        vol = miCaja1.volume();
        System.out.println("El volumen de miCaja1 es: " + vol);
        System.out.println("El peso de miCaja1 es: " + miCaja1.peso);
        System.out.println();

        vol = miCaja2.volume();
        System.out.println("El volumen de miCaja2 es: " + vol);
        System.out.println("El peso de miCaja2 es: " + miCaja2.peso);
    }
}
```

Objeto: miCaja1



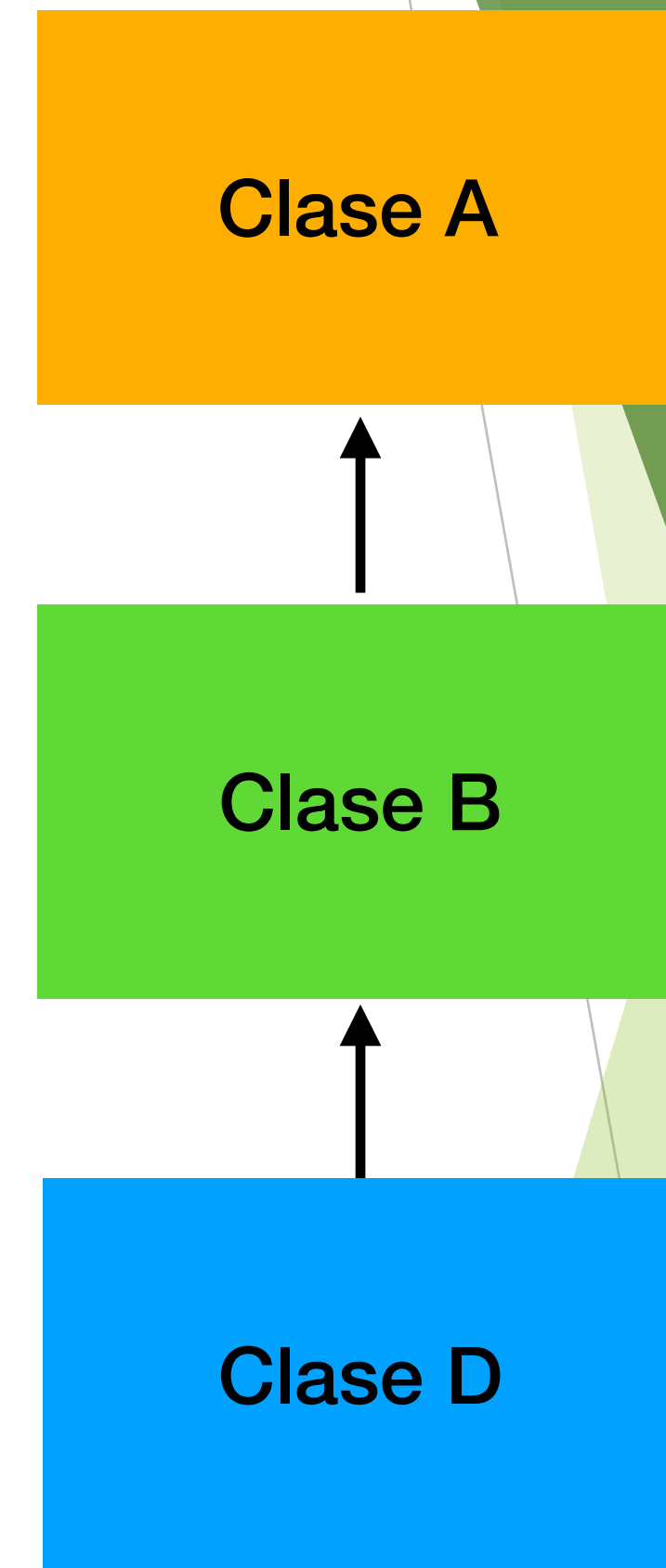
Salida del código

```
El volumen de miCaja1 es: 3000.0
El peso de miCaja1 es: 34.3

El volumen de miCaja2 es: 24.0
El peso de miCaja1 es: 0.076
```


Herencia multinivel

- Dadas tres clases llamadas A, B, y C, C puede ser una subclase de B, que es una subclase de A. Cuando ocurre este tipo de situación, cada subclase hereda todos los rasgos que se encuentran en todas sus superclases. En este caso, C hereda todos los atributos de B y A.



Multinivel

Herencia multinivel

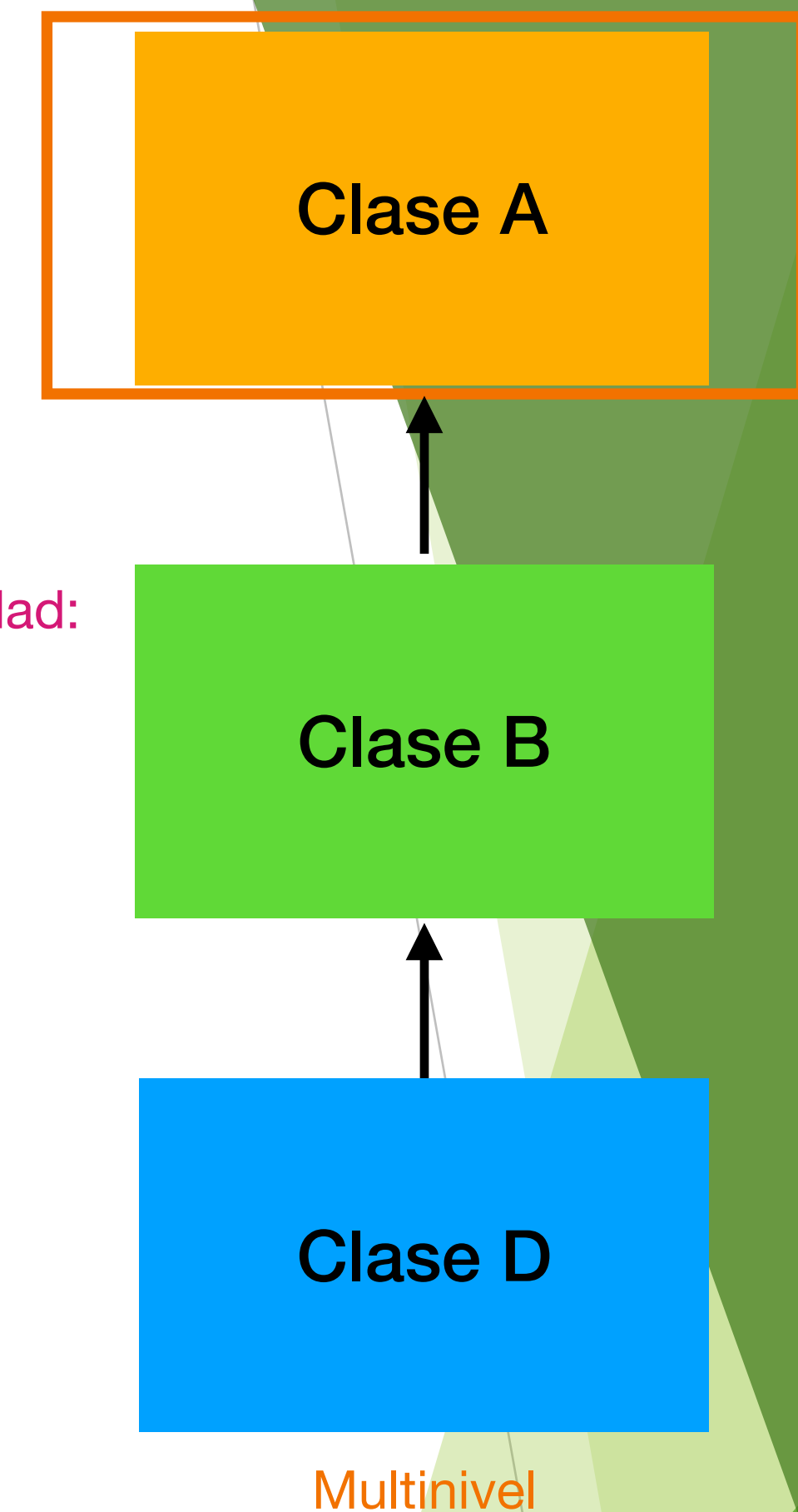
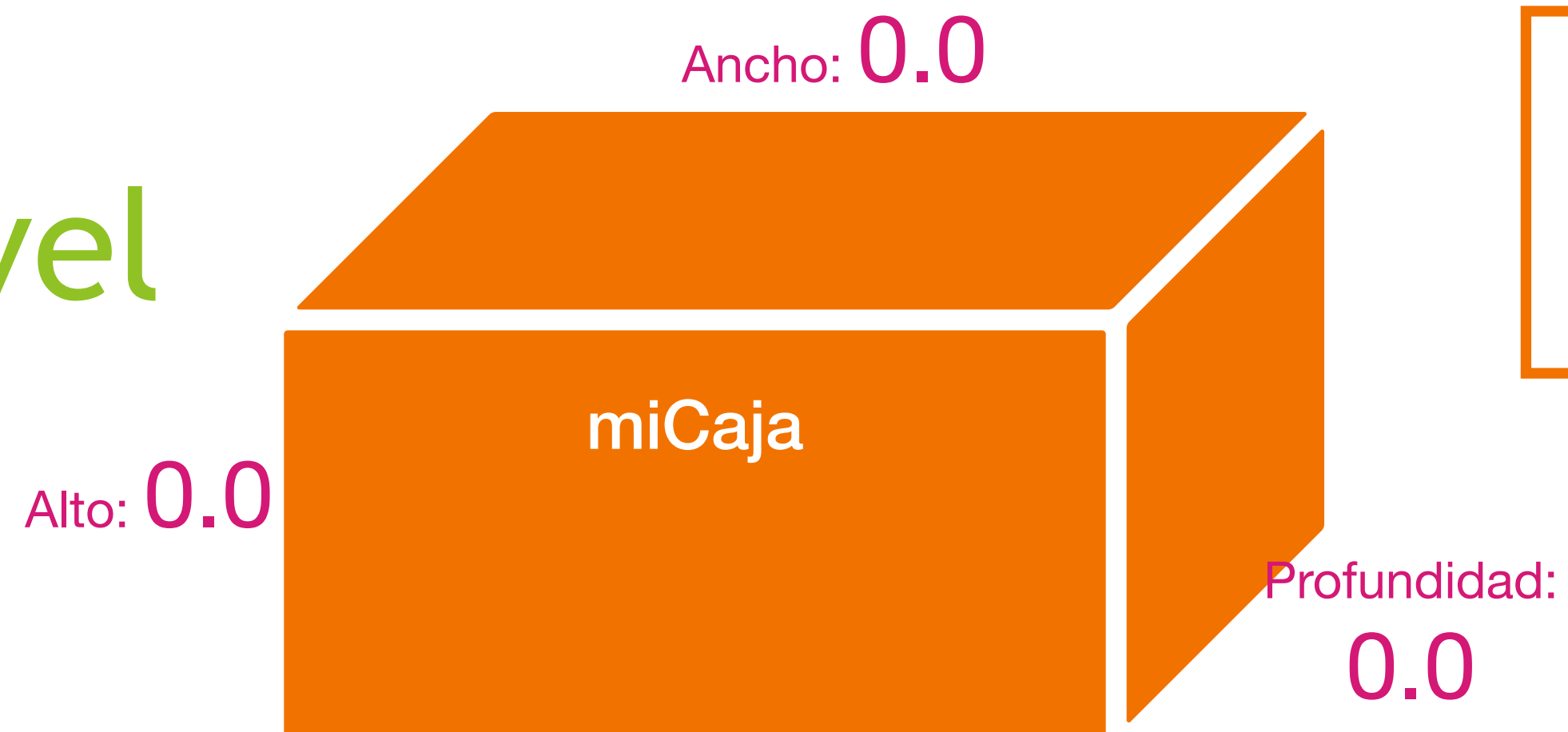
```
// Clase principal Caja o superclase
class Caja {
    private double ancho;
    private double alto;
    private double profundidad;

    // constructor usado cuando se especifican todas las dimensiones
    Caja(double a, double h, double p) {
        ancho = a;
        alto = h;
        profundidad = p;
    }

    // Constructor usado cuando no se especifican dimensiones
    Caja() {
        ancho = -1; // usar -1 para caja
        alto = -1; // sin inicializar
        profundidad = -1;
    }

    // constructor usado cuando el cubo es creado
    Caja(double tamano) {
        ancho = alto = profundidad = tamano;
    }

    // calcula y regresa el volumen
    double volumen() {
        return ancho * alto * profundidad;
    }
}
```



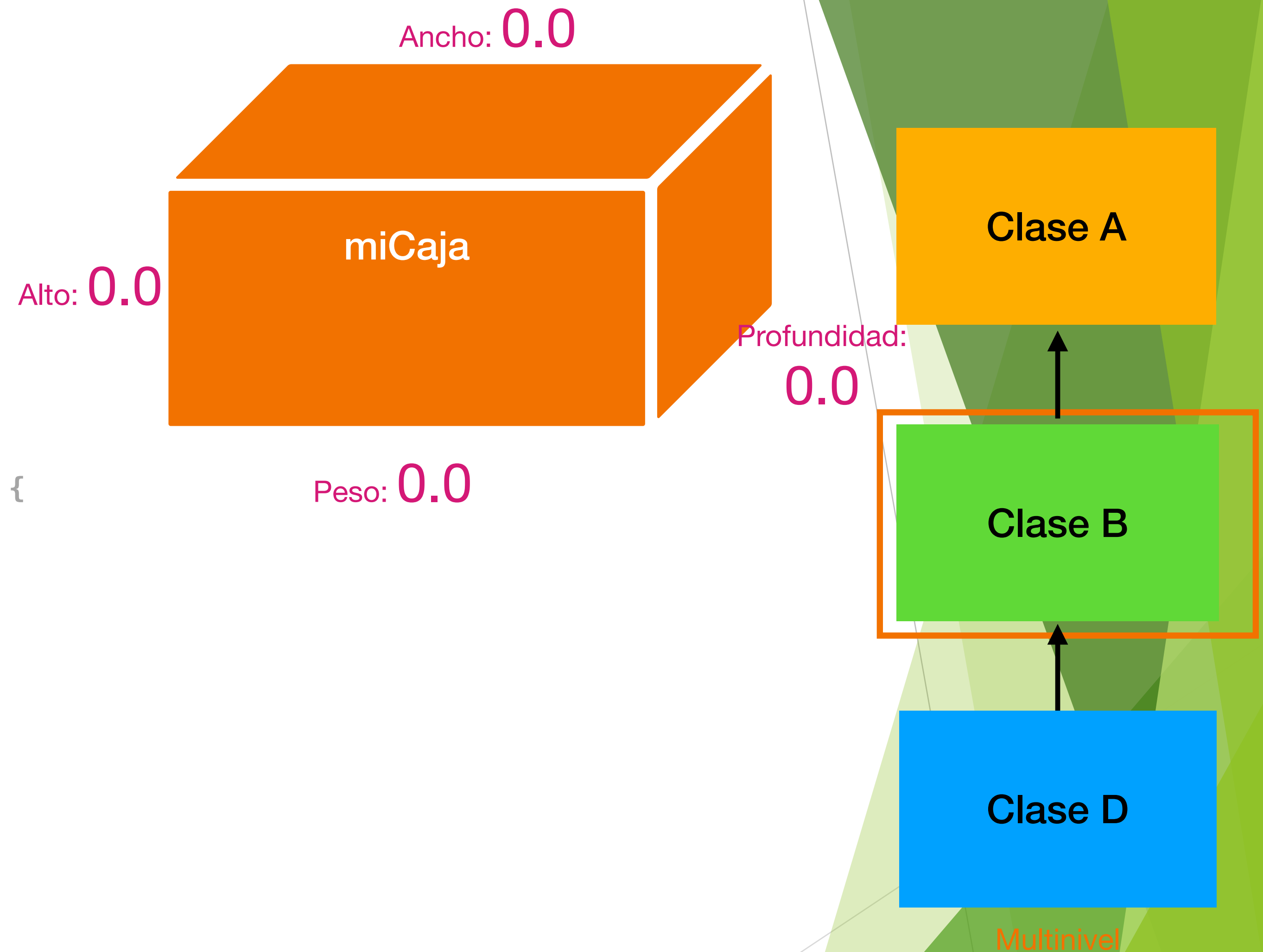
Herencia multinivel

```
//Caja es heredada para incluir el peso
public class PesoCaja extends Caja {
    double peso; // peso de la caja

    // constructor para PesoCaja
    PesoCaja(double w, double h, double d, double m) {
        super(w,h,d);
        peso = m;
    }

    // default constructor
    PesoCaja() {
        super();
        peso = -1;
    }

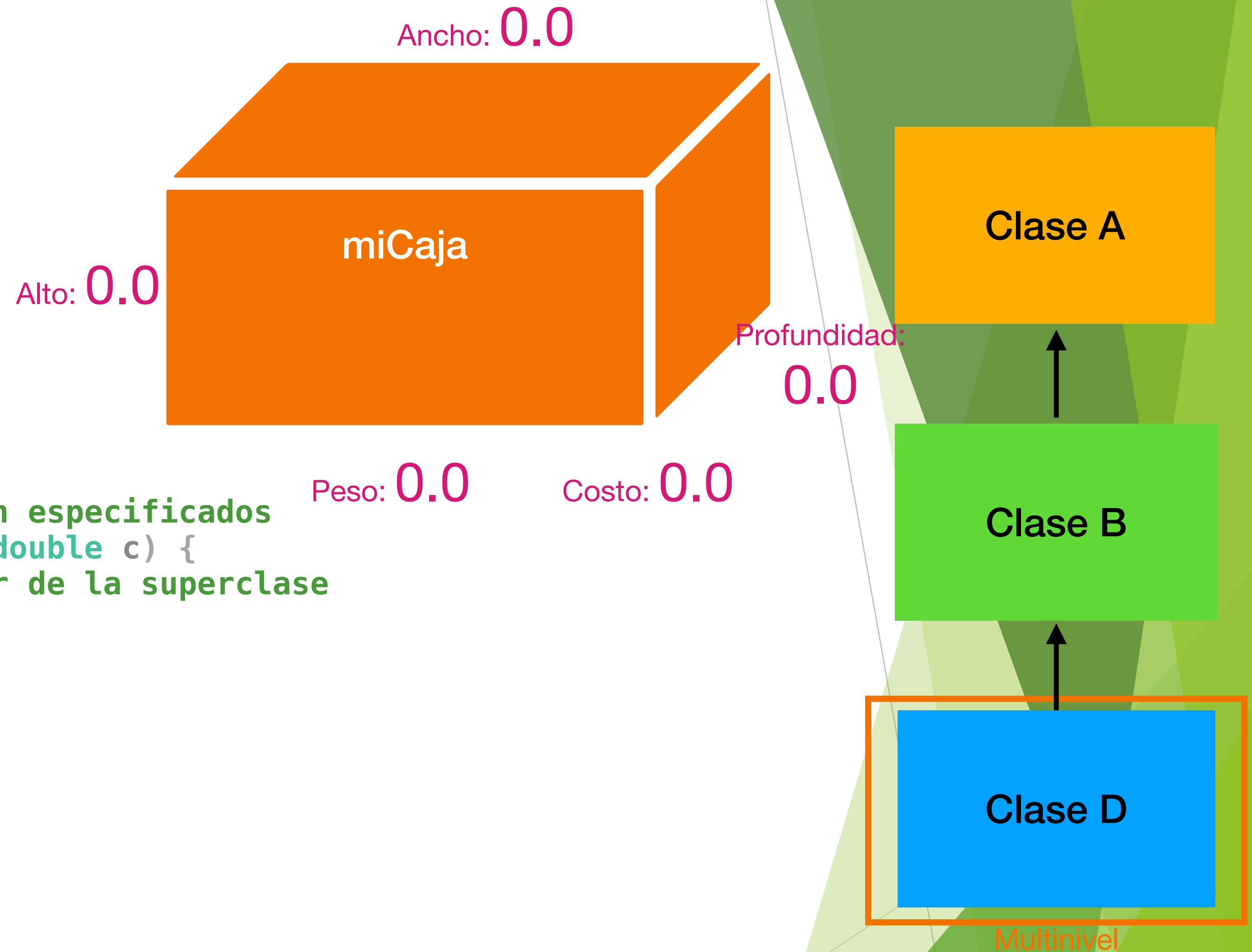
    // constructor usado cuando se crea un cubo
    PesoCaja(double len, double m) {
        super(len);
        peso = m;
    }
}
```



Herencia multinivel

```
// Agregar el costo de Envio .
public class Envio extends PesoCaja {
    double costo;

    // constructor cuando todos los parametros son especificados
    Envio(double w, double h, double d, double m, double c) {
        super(w, h, d, m); // llama al constructor de la superclase
        costo = c;
    }
    // constructor por default
    Envio() {
        super();
        costo = -1;
    }
    // constructor usado cuando un cubo es creado
    Envio(double len, double m, double c) {
        super(len, m);
        costo = c;
    }
}
```



Herencia multinivel

```
public class PruebaEnvio {  
    public static void main(String[] args) {  
        Envio envio1 = new Envio(10, 20, 15, 10, 3.41);  
        Envio envio2 = new Envio(2, 3, 4, 0.76, 1.28);  
        double vol;  
        vol = envio1.volume();  
        System.out.println("Volumen de Envio1 es " + vol);  
        System.out.println("Peso of Envio1 es "+ envio1.peso);  
        System.out.println("Costo de envio: $" + envio1.costo);  
        System.out.println();  
  
        vol = envio2.volume();  
        System.out.println("Volumen de Envio2 es " + vol);  
        System.out.println("Peso de Envio2 es "+ envio2.peso);  
        System.out.println("Costo de envio: $" + envio2.costo);  
    }  
}
```

Objeto: miCaja1

Alto: 20

Ancho: 10

miCaja

Profundidad: 15

Peso: 10

Costo: 3.41

Salida del código

```
Volumen de Envio1 es 3000.0  
Peso of Envio1 es 10.0  
Costo de envion: $3.41  
  
Volumen de Envio2 es 24.0  
Peso de Envio2 es 0.76  
Costo de envio: $1.28
```

Herencia multinivel

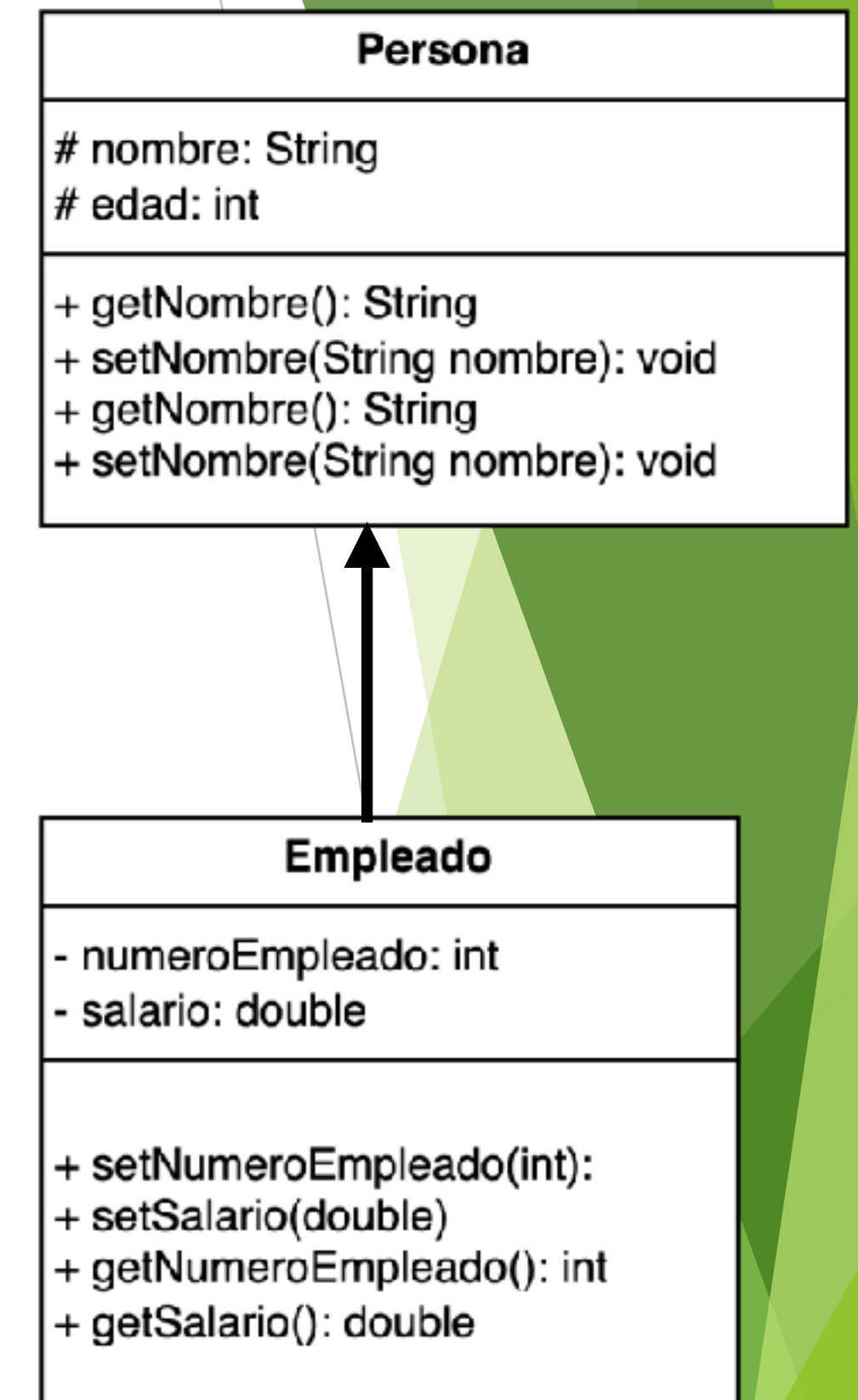
- ▶ Debido a la herencia. La clase Envío puede hacer uso de las clases previamente definidas de Caja y PesoCaja, agregando solo la información adicional que necesita para su propia aplicación específica. **Esto es parte del valor de la herencia; permite la reutilización de código.**
- ▶ El ejemplo anterior, ilustra otro punto importante: **super() siempre se refiere al constructor en la superclase más cercana.**
- ▶ El super() de Envío llama al constructor de PesoCaja. El super() en PesoCaja llama al constructor de Caja.
- ▶ En una **jerarquía de clases**, si un constructor de superclase requiere argumentos, entonces todas las subclases deben pasar esos argumentos “hacia arriba”. Esto es cierto tanto si una subclase necesita argumentos propios o no.

Ejemplo 3: Crear la superclase

- Tomemos un ejemplo simple. Suponga que ha definido una clase para representar a una persona de la siguiente manera

```
public class Persona {  
    protected String nombre;  
    protected int edad;  
  
    public Persona(String nombre, int edad) {  
        this.nombre = nombre;  
        this.edad = edad;  
    }  
    public void setNombre(String nombre) {  
        this.nombre = nombre;  
    }  
    public String getNombre() {  
        return nombre;  
    }  
    public void setEdad(int edad) {  
        this.edad = edad;  
    }  
    public int getEdad() {  
        return edad;  
    }  
}
```

- La clase Empleado es la subclase y la clase Persona es la superclase

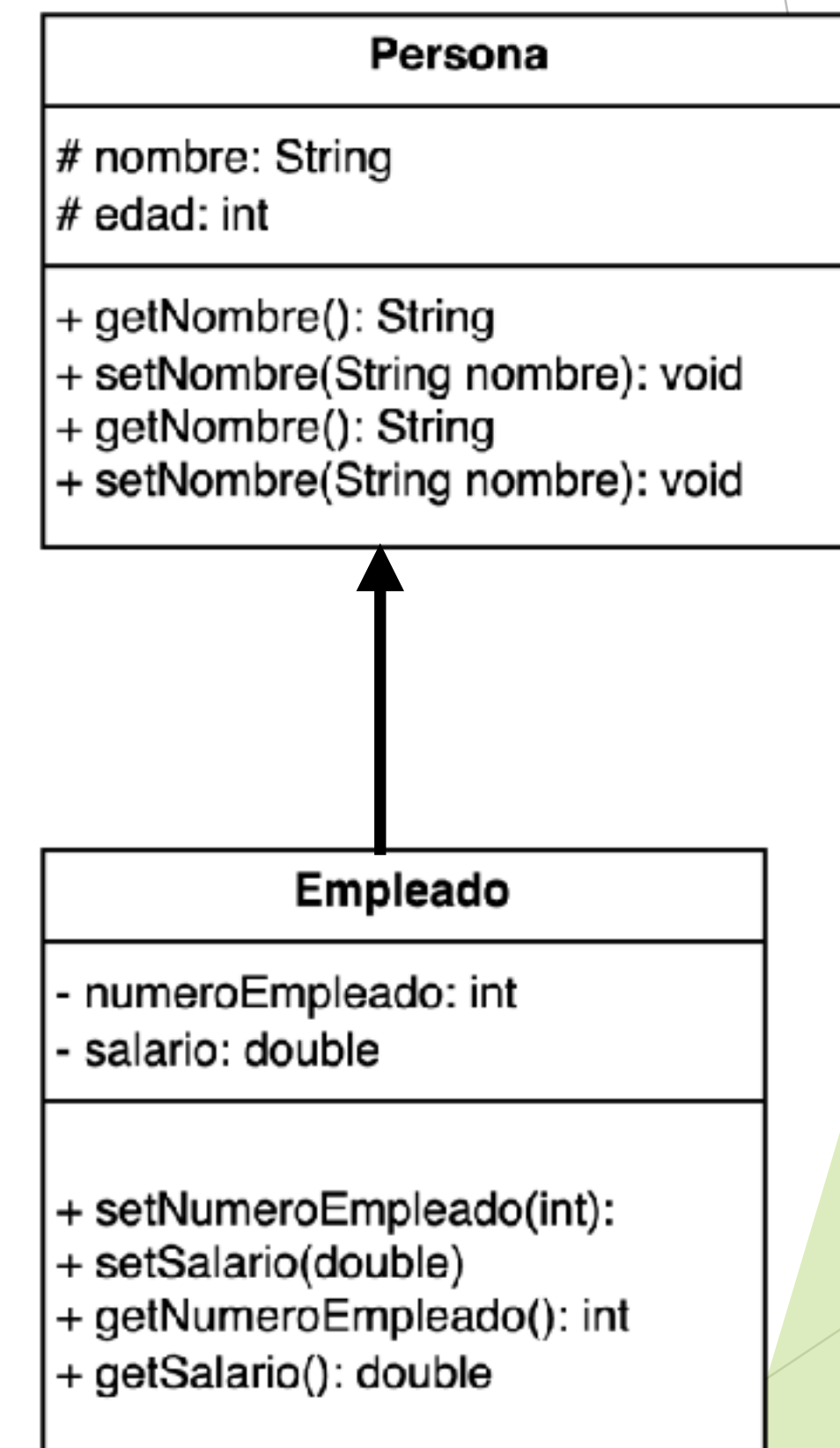


Ejemplo 3: Derivar una clase

- Ahora puede definir otra clase, basada en la clase Empleado, para definir empleados.

```
public class Empleado extends Persona {
    private int numeroEmpleado;
    private double salario;

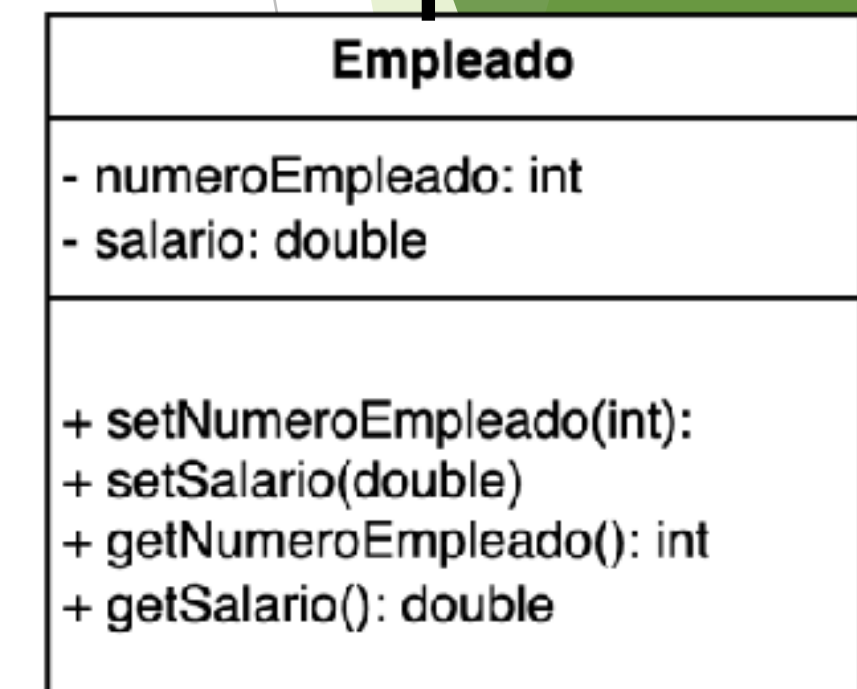
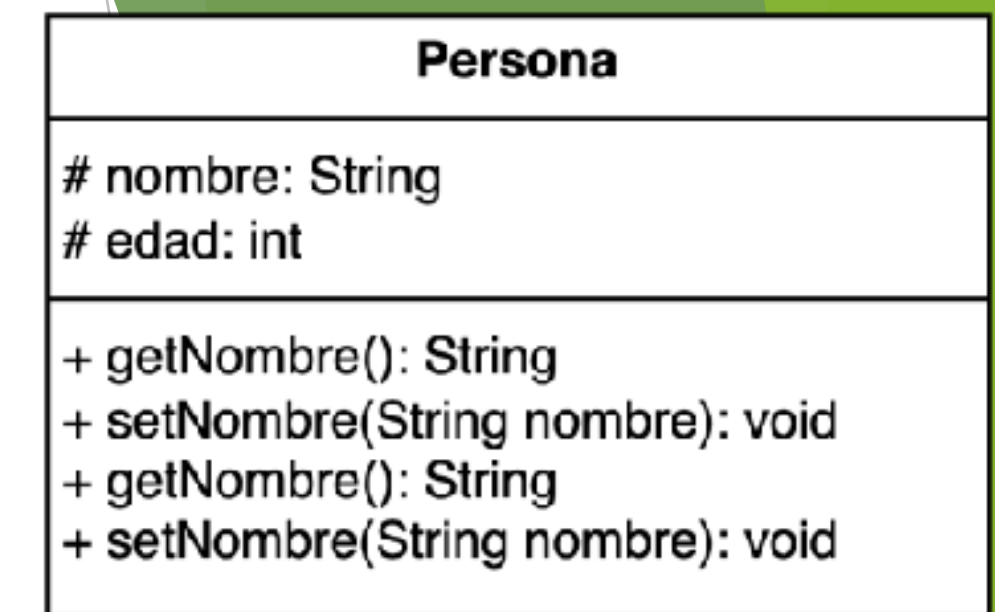
    public Empleado(String nombre,int edad, int numeroEmpleado, double salario) {
        super(nombre,edad);
        this.numeroEmpleado = numeroEmpleado;
        this.salario = salario;
    }
    public void setNumeroEmpleado(int numeroEmpleado) {
        this.numeroEmpleado = numeroEmpleado;
    }
    public int getNumeroEmpleado() {
        return numeroEmpleado;
    }
    public void setSalario(double salario) {
        this.salario = salario;
    }
    public double getSalario() {
        return salario;
    }
}
```



Ejemplo 3: Clase prueba

- Ahora puede definir otra clase, para probar la herencia

```
public class PruebaEmpleado {  
    public static void main(String[] args) {  
        Empleado empleado = new Empleado("Maria", 30, 26878, 450);  
        System.out.println(empleado.getNombre());  
        System.out.println(empleado.getEdad());  
        System.out.println(empleado.getNumeroEmpleado());  
        System.out.println(empleado.getSalario());  
    }  
}
```



Salida del código

```
Maria  
30  
26878  
450.0
```